> Have a look at balance_sheet_after_correction.pdf - a consolidated balance sheet from a japanese company.
Our task is to design a parser that converts each page, which can be even more complex format than the given balance sheet, into a correct table.

@AGENT.md

Also do an extensive research on these kind of industry standard parsers. Explain how they work and also tell me best choice for us to go ahead with.

We might as well need to find some other balance sheets or pages like these to test our parser in a later phase. So keep that in mind as well.
⎿ Read AGENT.md (60 lines)

● I'll start by looking at the workspace and the PDF.

Read 1 file, ran 3 shell commands (ctrl+o to expand)

● Born-digital PDF (real text layer, embedded Japanese fonts). Let me check how far PyMuPDF's built-in table finder and word coordinates get us before researching.

Ran 1 shell command (ctrl+o to expand)

● The built-in finder gets the body but drops the column header row and the indent hierarchy (4 clean indent levels at x≈75/88/101/114). Now the research on industry parsers.

● Web Search("PDF table extraction benchmark 2026 Docling Marker MinerU PaddleOCR TEDS comparison")
⎿ Did 1 search in 7s

● Web Search("Japanese financial statements PDF table extraction 決算短信 貸借対照表 parser")
⎿ Did 1 search in 6s

● Web Search("camelot pdfplumber lattice stream table extraction algorithm how it works")
⎿ Did 1 search in 7s

● Web Search("EDINET API XBRL download balance sheet TDnet 決算短信 XBRL")
⎿ Did 1 search in 7s

● Web Search("Docling TableFormer architecture layout model table structure recognition how it works")
⎿ Did 1 search in 5s

● Web Search("Azure Document Intelligence vs AWS Textract vs Google Document AI table extraction Japanese accuracy 2026")
⎿ Did 1 search in 6s

● Web Search("YomiToku Japanese document AI table recognition license accuracy")
⎿ Did 1 search in 5s

● Web Search("FinTabNet Table Transformer PubTables-1M financial table structure recognition hierarchical row headers")
⎿ Did 1 search in 5s

● Web Search("TDnet 決算短信 XBRL 添付 財務諸表 iXBRL Attachment 貸借対照表 tse-acedjpfr")
⎿ Did 1 search in 6s

● Web Search("vision language model financial table extraction hallucination numbers PDF hybrid text layer validation 2026")
⎿ Did 1 search in 7s

● Web Search("pdfplumber table settings lines text strategy explicit vertical lines how find_tables works PyMuPDF")
⎿ Did 1 search in 7s

● Fetch(https://needl.ai/technical-notes/benchmarking-pdf-extraction-tools)
⎿ Received 77KB (200 OK)

● Fetch(https://github.com/opendatalab/OmniDocBench)
⎿ Received 734.5KB (200 OK)

● Advising using Opus 5.5
⎿ √ Advisor has reviewed the conversation and will apply the feedback

● Let me verify two of those observations (subtotal rounding and underline marks) directly before writing them down.

Ran 1 shell command (ctrl+o to expand)
● Both checks confirmed: current-year sums are off by 2-3 due to 千円 rounding (so validation needs tolerance), and the underlines are word-width rules unique to the current-year column, distinguishing them from full-column subtotal rules. Now I'll write up the research doc.

● Write(docs¥research¥01-table-parser-landscape.md)

```
└ Wrote 232 lines to docs¥research¥01-table-parser-landscape.md
     1 # Research: PDF Table Parsers for Japanese Financial Statements
     2
     3 > Status: research only. No parser code has been written yet.
       tion.pdf`
     5
     6 ---
     7
     9
    10 `balance_sheet_after_correction.pdf` is page 4 of a corrected 決
       算短信 (Kessan Tanshin, "(訂正後)"),
   … +332 lines (ctrl+o to expand)
```

● I've written the research doc to docs/research/01-table-parser-landscape.md. Per AGENT.md I haven't written any parser code or installed anything. I need a few answers from you before I propose the implementation plan.

What this page needs to parse correctly

The PDF is page 4 of a corrected earnings summary (決算短信, marked 訂正後), showing the assets half of a consolidated balance sheet. The page has a real text layer, so every character can be read exactly and no OCR is needed. Getting a correct table means solving these:

- No vertical lines. The only drawings are row shading and horizontal rules, so the columns have to come from where the text lines up.
- Hierarchy from indentation. The labels sit at five indent levels (x ≈ 62/75/88/101/114). その他 appears 4 times and 減価償却累計額 3 times. A flat table can't tell them apart, so each row needs its parent path.
- Value conventions. △ means negative; ― means nil, which is not the same as 0; △0 has to survive as-is; the unit is 千円; some characters are full-width.
- Correction marks. 218,836 and 1,123 are underlined by rules exactly as wide as the number, which in a corrected filing usually marks the corrected cells. They're easy to tell apart from subtotal rules, so the parser can keep them as a flag on each cell.
- The table continues. The liabilities and equity half is almost certainly on the next page, so tables must span pages.
- Subtotals don't add up exactly. The current assets items sum to 5,863,681 against a stated 5,863,684, because each item is rounded to 千円. Checking totals needs a small tolerance.

As a baseline, PyMuPDF's built-in find_tables() gets all the body values right. It loses the column header, the hierarchy and the underline flags.

How the parsers out there work

Approach: Rule-based, from the text layer
Examples: Camelot, pdfplumber, PyMuPDF
How it works: Reads characters with their positions; builds the grid from ruling lines or from text alignment
Weakness for us: Fails on scans; needs tuning per layout

Approach: Deep-learning pipeline
Examples: Docling, MinerU, PaddleOCR, Table Transformer, YomiToku
How it works: One model finds the table, a second predicts rows, columns and spans, then text is taken from the PDF or OCR
Weakness for us: Doesn't output indentation hierarchy; trained mostly on papers

---

Approach: Vision-language models (VLMs)
Examples: PaddleOCR-VL, MinerU 2.5, Gemini, Claude
How it works: The page image goes in and the model writes out the table
Weakness for us: Can change or invent numbers; needs a GPU or a paid API

Approach: Cloud APIs
Examples: Azure, Textract, Google Document AI
How it works: Hosted versions of the above
Weakness for us: Cost; documents leave your machine; no Japanese benchmarks

Be careful with the published rankings. No benchmark covers Japanese balance sheets, and none checks whether every number is exactly right. Several are run by vendors or test a single document. The doc rates each source.

Recommendation

Read the text layer first, fall back to a model only when that fails, and use the accounting arithmetic to decide which answer to trust:

1. For born-digital PDFs, build the table from PyMuPDF's words and drawings. It can't invent a digit.
2. For scanned or garbled pages, fall back to Docling (permissive license, runs on CPU). PaddleOCR-VL is the stronger option if you have a GPU.
3. Check every result against the accounting identities: line items sum to their total (within rounding), gross minus accumulated depreciation equals net, total assets equals total liabilities plus equity.
4. Output hierarchical JSON with per-cell flags, plus a flat CSV view.

Finding more test pages

Companies also file these statements as XBRL (tagged machine-readable data), which gives us the correct value of every cell to score against automatically:
- EDINET API v2 has annual-report PDFs plus their XBRL. It's free, but you need to register for an API key; I can't do that for you.
- TDnet publishes earnings-summary PDFs with XBRL. Its public listing only keeps about 31 days, so we'd need to collect regularly.

The harder cases to collect are listed in section 6 of the doc. The toughest is the statement of changes in equity (株主資本等変動計算書), a wide grid with multi-level headers.

Questions before I write the implementation plan

1. Commercial or closed-source use? PyMuPDF is AGPL; if that's a problem I'd use pdfplumber (MIT) instead. Some model licenses (YomiToku, Marker, MinerU) also depend on this.
2. Are scanned PDFs in scope for phase 1? If not, the model fallback can wait.
3. Are GPUs, cloud APIs or hosted AI models allowed, given these are financial documents?
4. Output format: hierarchical JSON with flags (my recommendation), flat CSV, Excel, or several?
5. Do you have the next page (liabilities and equity) or the 訂正前 version of this one? Both would be immediate test cases.

Sources

- Camelot: how it works (https://camelot-py.readthedocs.io/en/latest/user/how-it-works.html)
- pdfplumber docs (https://docsearch.algolia.com/mcp/docs/repo/jsvine/pdfplumber)
- Docling technical report (https://export.arxiv.org/pdf/2408.09869)
- PubTables-1M (https://arxiv.org/pdf/2110.00061) and Table Transformer (https://github.com/microsoft/table-transformer)
- OmniDocBench (https://github.com/opendatalab/OmniDocBench) and its paper (https://arxiv.org/html/2412.07626v2)
- Dr. DocBench (https://arxiv.org/pdf/2606.01393)
- Needl.ai benchmark (https://needl.ai/technical-notes/benchmarking-pdf-extraction-tools)
- Independent 27-document benchmark (https://github.com/hraju115/pdf-extraction-benchmark) and write-up (https://dev.to/harshith_raju_6ba0e1f9a52/i-benchmarked-8-pdf-extractors-on-27-documents-from-1-to-487-pages-heres-what-broke-and-what-won-ca8)
- Docling vs Marker vs MinerU (https://adityamangal98.medium.com/docling-vs-marker-vs-mineru-the-ultimate-open-source-pdf-parser-benchmark-2026-which-is-best-a36ecbb6c6b1)
- READoc (https://arxiv.org/pdf/2409.05137)
- Reducto on Azure Document Intelligence (https://reducto.ai/blog/evaluating-azure-document-intelligence-for-pdf-parsing)
- Cloud OCR comparison 2026 (https://imagetotable.ai/blog/google-vs-aws-vs-azure-ocr-2026)
- French financial-table VLM benchmark (https://arxiv.org/abs/2602.10384)
- Deterministic financial PDF extraction (https://apis.hyperbots.com/blogs/deterministic-extraction-from-financial-pdfs)
- YomiToku (https://docsearch.algolia.com/mcp/docs/repo/kotaro-kinoshita/yomitoku)
- Docling models (https://www.promptlayer.com/models/docling-models)
- TDnet file specification (https://www.jpx.co.jp/equities/listing/disclosure/xbrl/nlsgeu000005vk0b-att/File_Specification_for_TDnet_Filing_20240721.pdf)
- TSE notice on TDnet/EDINET XBRL (https://www.nse.or.jp/company/com/notice/files/h19zikig10.pdf)
- EDINET API tutorial (https://zenn.dev/joanofarc/articles/get-financial-info-by-edinet)
- TDnet disclosure types (https://edinetdb.jp/blog/tdnet-disclosure-types)

> /model sonnet
  ⎿ Set model to claude-sonnet-5-5

> I have few answers to your questions and also a few questions of my own. first to answer your questions:

1. No it is not used for any commerical purposes. It is not necessarily closed source either. Think of it as a parser we are building for personl use. Which parser / library has proven to be more accurate and better btw? or which is the typical choice for industrial use?
2. As of now we most probably will not be dealing with the scanned pdfs. However in case we do have can we not build a hybrid parser which will route the task to an OCR based parser incase it fails normally?
3. As long as the resource requirement is not high.
4. we need to properly format it as tables after parsing. So I think CSV or Excel might be a more appropriate choice here.
5. After some searching I found out that this page is very similar to this pdf most probably: https://assets.minkabu.jp/news/article_media_content/urn%3Anewsml%3Atdnet.info%3A20260609565929/140120260609565929.pdf?

Maybe we can look for more in - https://www.yamato-in.co.jp/ir/

I have downloaded it in the root folder as well. check complete_pdf.pdf

My questions to u are:
- If we are going to consider that all the pages may have same layout - we

can surely build a rule based parser right?
- Do u think a hybrid parser would be able to produce much better output. for instance something which fall backs to other methods incase of a failure.
- What are the metrics we can use to evaluate the performance / correctness / accuracy of the parser? How is the correctness defined for the parsers?

If i say we have no restraints, then what would you say is the best option for us to go with? VLM? Also explain why?

● Let me look at the full PDF before answering, so the answers rest on what's in it.

Read 1 file, ran 2 shell commands (ctrl+o to expand)

● I read all 19 pages of complete_pdf.pdf before answering. It is the same company's 決算短信 (ヤマト モビリティ & Mfg., 7886).

- The two PDFs differ in exactly 3 cells. complete_pdf.pdf is the uncorrected original. Compared with balance_sheet_after_correction.pdf, the corrected page has 218,836 where the original has 53,925, 1,123 where it has 166,033, and △0 where it has 0. Those are the underlined cells, so the underline does mark corrections. You now have a ready-made test: a correct parser should report exactly those three differences.
- There are 12 table pages. They are the balance sheet (pp. 8-9), income statement and comprehensive income (10-11), equity statement (12-13), cash flow (14-15) and segment notes (17-19).
- The layout varies by page. The equity statement (p. 12) has full grid lines and a two-level header, unlike the balance sheet. Four of the table pages are also ones where PyMuPDF's find_tables() returned multiple or odd tables.
- Page 1 contains images, but it's a cover notice, not a table.

Your answers

1. Which library is most accurate, and what is the industry choice?
- For a text-layer PDF like this one, accuracy of the characters is not the differentiator. PyMuPDF, pdfplumber and pdfminer all read the same embedded characters. The difference is in how they find the table structure, and none of them handles hierarchy or the △ and ー conventions.
- What industry uses: I only have indirect evidence. Enterprises processing mixed documents lean on Azure Document Intelligence or AWS Textract, or on Docling and MinerU for open source. For one standardised template, the usual approach is a custom extractor with accounting checks. I found no benchmark that settles "best" for Japanese financial statements.
- Licensing: for personal, non-distributed use, PyMuPDF's AGPL is not a practical problem. I'd keep PyMuPDF.

2. Can the parser fall back to OCR? Yes. That is the triage step already in the plan. The cheap test is whether the text layer is healthy (valid CJK and digit characters, no garbled text). If it isn't, route the page to OCR or Docling. It can wait until you actually meet a scan.

3. Resource limits: with a low-resource requirement, the text-layer path runs on CPU in milliseconds per page. Docling can run on CPU as a fallback.

4. Output: Excel, with one sheet per statement and an extra column for hierarchy depth. The parser would carry the hierarchy and flags internally, and Excel would be a rendering of them (indented labels, red for negatives, highlighted corrected cells). CSV works for single tables but loses formatting.

Your questions

If all pages share a layout, can we build a rule-based parser? Yes, and for this document I'd recommend it. Two caveats:
- "Same layout" means the same kind of page, not the same page. Even within this PDF the balance sheet has no vertical lines, the equity statement has a full grid, and the segment pages differ again. Hard-coding rows would be brittle.
- So build a generic engine (words, rules, column clustering, indentation) with a small per-statement-type config, such as expected header depth or the accounting checks to run. That reuses across companies and years.

Would a hybrid parser give much better output? Yes, but only if you can tell when the primary path failed. The accounting identities (line items sum to the total within rounding, gross minus accumulated depreciation equals net, total assets equals total liabilities plus equity) catch most errors on statements. They don't cover notes or segment tables. For those I'd also check that the column count is consistent across rows and that no cell is left empty unexpectedly. Beyond those checks, a hybrid is only as good as its failure detector.

How is parser correctness measured? There are three layers.

Against ground truth (accuracy):
- Cell exact match (precision, recall, F1), keyed by row path, column and value. This is the headline metric. For balance sheets I'd report it, plus the share of pages where every cell is right.
- Numeric exactness after normalisation (△, ー, full-width digits, units). A single wrong digit is a failure here.
- Hierarchy accuracy: does each row have the right parent path?

- Structure scores from the literature: TEDS and GriTS (the metric from the PubTables-1M paper). They are useful for comparing against published results, but too forgiving for finance, where one wrong digit scores almost perfectly.
- Flag accuracy for correction underlines and nil versus zero.

Without ground truth (self-consistency):
- Pass rate of the accounting checks, and the number of cells where two extractors disagree.

Where ground truth comes from: the XBRL filed with the same document gives exact values automatically. Failing that, hand-annotate a few pages. For this PDF you can start with the paired-diff test above.

If there were no constraints, would a VLM be best?

No, not on its own, even with no constraints. My reasoning:

- A VLM reads pixels, so it can alter digits. In the one financial-table benchmark I found, GPT-5.5 found every table but changed values. That evidence is thin: it's one 17-page document, and the judge was also one of the models tested. The mechanism still holds, since the model re-generates the numbers instead of copying them.
- This PDF already contains the exact numbers. Copying them out is strictly safer than re-reading them.
- The best option with no constraints is two independent readers plus a judge. The deterministic text-layer parser is the primary. A strong VLM reads the same page image as a second opinion. Wherever the two disagree, or an accounting check fails, flag the cell for review. Two independent extractors rarely make the same error, so agreement is strong evidence.
- The VLM's real value is on pages where rules struggle (odd layouts, scans) and as the safety net that catches a bug in my rules.
- This is a judgement from the evidence I gathered, not a measured result on your documents. We can test it directly: run both on these 12 pages and compare against the diff and the checks.

If you only want the data, the filing's XBRL is more reliable than any PDF parse. For the parser itself it serves as the answer key.

Next step

Per AGENT.md I won't write code until you approve a plan. Two things would help me scope it:
1. Scope for phase 1: all 12 table pages, or start with the balance sheet and income statement and grow from there?
2. Should I write the implementation plan now (architecture, modules, dependencies, test layout, timeline, risks), using this document as the test set?

> does the complete pdf follow a same layout? Can we build one rule based parser for all of the pdf?
if we can build a parser for phase 1 then give me the implementation plan for the same as well

● I'll check each table page rather than guess. First I'll measure each page's structural features, then look at the ones that seem different.

Read 3 files, ran 2 shell commands (ctrl+o to expand)

● Let me recheck the underline evidence—page 10 also shows underlined numbers, which doesn't fit my assumption that underlines only mark corrections.
· summary

Ran 2 shell commands (ctrl+o to expand)

● Correcting my earlier claim: underlines don't consistently mark corrections—page 10 has unrelated underlines, and △0 was changed without one. I'll update the research doc to reflect this. · summary

● Update(docs¥research¥01-table-parser-landscape.md)
└ Added 1 line, removed 1 line
```
24  | Unit | `（単位：千円）` above the table | Store unit/scale as
     table metadata |
25  | Full-width chars | `３月31日`, `（純額）` | NFKC normalisatio
    n |
26  | Subtotal rules | Black rules spanning full column width above
     合計 / 純額 rows | Useful signal for "total" rows |
27 -| **Correction marks** | `218,836` and `1,123` (current year) h
   -ave a rule exactly as wide as the word (31.6 pt / 22.6 pt) | In
   - a 訂正後 document underlines mark corrected cells; keep as a p
   -er-cell flag |
27 +| **Word-width underlines** | `218,836` and `1,123` (current ye
   +ar) have a rule exactly as wide as the word (31.6 pt / 22.6 pt)
   +. These two are corrections vs the original (53,925 / 166,033),
   + but the 3rd correction (`△0`, was `0`) has no underline, and t
   +he *original* filing also has such underlines on p.10 (e.g. 187
   +,285) where nothing was corrected | **Not a reliable correction
   + marker.** Record as a low-trust per-cell flag only; detect cor
   +rections by diffing 訂正前/訂正後 |
28  | Page furniture | `- 4 -`, `(訂正後)`, section titles 3. / (1
```

```
    ) | Strip from the table, keep as metadata |
29  | Continuation | Liabilities & equity (負債純資産の部) is almos
    t certainly on the next page | Design must support multi-page t
    ables |
30
```

● Update(docs¥research¥01-table-parser-landscape.md)
```
  └ Added 1 line, removed 1 line
    208  - キャッシュ・フロー計算書 (cash-flow statement)
    209  - 有報 side-by-side layouts, footnote markers (※1, ※2), "注記
         番号" note columns
    210  - IFRS filers (different account names, 4-column formats), ban
         ks/insurers (different templates)
    211 -- 訂正前 vs 訂正後 pairs (test the underline/correction flag)
    211 +- 訂正前 vs 訂正後 pairs (this PDF + `complete_pdf.pdf` differ
        + in exactly 3 cells: a ready-made diff test)
    212  - Scanned or image-only filings, and PDFs with broken ToUnicod
         e maps (garbled text)
    213  - Landscape pages, two tables on one page, tables split across
          columns
    214
```

● Update(docs¥research¥01-table-parser-landscape.md)
```
  └ Added 1 line, removed 1 line
    176     cannot invent a digit; a VLM can.
    177  2. **Balance sheets are self-checking.** The accounting identi
         ties let the system detect its own
    178     errors and choose between extractors, instead of relying on
          a benchmark score.
    179 -3. **The hard parts are domain-specific** (indent hierarchy, △
        -, ―, underlines, multi-page continuation).
    179 +3. **The hard parts are domain-specific** (indent hierarchy, △
        +, ―, multi-page continuation).
    180     No off-the-shelf tool outputs these; we need our own layer
         on top regardless of the backend.
    181  4. **The fallback is pluggable.** Docling is the safest defaul
         t (MIT, CPU-capable, uses the text layer).
    182     A specialised VLM (PaddleOCR-VL) is the strongest choice fo
         r scans if a GPU is available.
```

● Update(docs¥research¥01-table-parser-landscape.md)
```
  └ Added 1 line, removed 1 line
    40  Strict equality would raise false alarms.
    41
    42  Baseline: `page.find_tables()` (PyMuPDF) finds 1 table, 43 rows
          × 3 cols, with correct body values.
    43 -It loses the header, the hierarchy and the underline flags. It
       -is a good start but not a correct table.
    43 +It loses the header and the hierarchy. It is a good start but n
       +ot a correct table.
    44
    45  ---
    46
```

● Now the plan document.

● Write(docs¥plans¥phase1-implementation-plan.md)
```
  └ Wrote 237 lines to docs¥plans¥phase1-implementation-plan.md
    1 # Phase 1 Implementation Plan: Financial-Statement Table Parser
    2
    3 > Status: **PROPOSED, awaiting approval.** No code has been writt
    4 > Test corpus: `complete_pdf.pdf` (19 pages, original) + `balance
      _sheet_after_correction.pdf` (1 page, corrected).
    5 > Background: [../research/01-table-parser-landscape.md](../resea
      rch/01-table-parser-landscape.md)
    7 ---
    8
    9 ## 1. Layout survey of `complete_pdf.pdf` (basis for the scope)
   … +319 lines (ctrl+o to expand)
```

● Update(docs¥plans¥phase1-implementation-plan.md)
```
  └ Added 2 lines, removed 1 line
    187  1. **Unit tests:** `parse_value` (△, ―, `△0`, full-width, com
         mas), level clustering, row typing, tolerance logic.
    188  2. **Integration tests:** each Class A page parses to the expe
         cted number of rows, columns and header text.
    189  3. **Diff test (free ground truth):** parse the corrected page
          and the original p. 8, and diff the tables. Expected result i
         s *exactly* these 3 cells:
    190 -   `投資有価証券` current 53,925 -> 218,836; `関係会社株式`...
        - see note; `貸倒引当金` current 0 -> △0. (The cell mapping is
        -confirmed in implementation, not assumed here.)
    190 +   `投資有価証券` (current year) 53,925 -> 218,836; `関係会社
        +株式` (current year) 166,033 -> 1,123; `貸倒引当金` (current y
        +ear) 0 -> △0.
    191 +   Both versions also pass the 投資その他の資産合計 sum check
        +within rounding (369,360 and 369,361 vs stated 369,362), so th
        +e oracle works on both.
    192  4. **Accounting checks** as a correctness oracle on all 6 page
         s.
```

```
   193  5. **Golden files:** one reviewed JSON per page. You review on
        e page by hand, then these stay as regression tests. Golden fi
        les made by the parser alone would be circular, so the
   194     independent signals are the accounting checks, the diff tes
        t, and your spot check.
```

● No, the PDF doesn't follow one layout. A single rule-based package can still cover all the table pages, but it needs more than one extraction strategy. I've written the Phase 1 plan to docs/plans/phase1-implementation-plan.md. I haven't written any code or installed anything.

Does the PDF share one layout?

I measured all 12 table pages in complete_pdf.pdf. They fall into three classes:

Class: A. Banded statement
Pages: 8, 9, 10, 11, 14, 15
Content: Balance sheet, income statement, comprehensive income, cash flow
Structure: Blue and white row bands, no vertical lines, 3 columns, indentation hierarchy

---

Class: B. Ruled matrix
Pages: 12, 13
Content: Statement of changes in equity
Structure: Full grid, two stacked blocks per page (6 and 9 columns), two-level spanning headers, two-line row labels

---

Class: C. Ruled notes tables
Pages: 17, 18, 19
Content: Segment and per-share information
Structure: Full grid, spanning headers, prose cells, footnotes, tables continuing across pages

- Class A is very regular. All six pages share the same bands, column positions and indent step, so one extractor can handle them.
- Classes B and C come from the grid lines, not from text alignment, so they need a different strategy.
- My answer to "one rule-based parser for all pages": yes, if it is one package with a shared core and two strategies (banded and ruled-grid) picked per page. One heuristic covering all three classes would be fragile.

Correction to what I told you earlier

I said the underlines mark corrected cells. That was too strong:
- In the corrected page, 218,836 and 1,123 are underlined and are real corrections. The original had 53,925 and 166,033.
- The third correction, △0 (originally 0), has no underline.
- The original filing also has word-width underlines on page 10 where nothing was corrected.

I've fixed the research doc. The parser will keep the underline only as a low-trust flag. Corrections are detected by diffing the 訂正前 and 訂正後 versions. That diff gives exactly 3 cells, a ready-made test.

Phase 1 plan

Scope: Class A pages (6 pages plus the corrected page), producing validated Excel and CSV output.

Method:
1. Each blue or white band is exactly one row, which is more reliable than guessing rows from text positions.
2. The header is rebuilt from the words above the first black rule, which recovers the row find_tables() dropped.
3. The columns come from the right edges of the numbers.
4. Each row's hierarchy comes from its indentation, so you get a parent path like 資産の部 > 固定資産 > 有形固定資産 > その他.
5. Values are normalised: △ becomes negative, ― becomes nil, △0 is kept as written, and full-width digits are converted.
6. Totals are checked against the sum of their children, allowing a tolerance for rounding to 千円.

Testing:
- Unit tests.
- The corrected-versus-original diff test.
- Accounting checks on all six pages.
- A page of golden values that you review by hand.

Effort: about 5 days. The doc also lists the architecture, code snippets and a risk table.

Decisions I need from you

1. Scope: Class A only for Phase 1 (my recommendation), or include the grid pages now? The equity statement is the hardest page in the file.
2. Packages: may I install openpyxl (Excel output), pytest, ruff and mypy? PyMuPDF, pandas and PyYAML are already installed.

3. Layout: are you happy with source code under src/, tests under tests/, and
   one Excel workbook per PDF with a sheet per statement?

One limitation: I have no independent answer key for these pages yet, such as
XBRL. Until we get one, correctness rests on the accounting checks, the diff
test and your manual review of one page.

> Ok for the first phase ew can surely just work on developing a parser for the
balance_sheet_after_correction.pdf
Let's try to get the parser to be as good as possible for this case.

for your questions:
1. I have talked about this above.
2. yes, install all the necessary packages.
3. yes. it is a modular code maintainence option.
4. one workbook per PDF with one sheet per statement, OK? - Ok.

And once the parser is made - test it out on the page and give me the
relevant metrics showing the performance of the parser. We do not have gound
truth so you can generate it as well. I don't think it will hard for you.

Also tell me - You have mentioned about how the AWS or some cloud parsers are
really good. What about them? do u think we shall use it?
Can we not create a testing pipeline or algorithm to tally the balance sheet
- Like there are usually some rules a balance sheer must follow (Assests =
liabilities + Equity). So once we have parsed things we can check for this
and this will really help is identifying where and what might have gone wrong
right? So I think we can create a algo which can check this from excel
sheet, which is our output and has follows a same format. LMK what u think
later!

● Approved. I'll start by installing the packages and then inspecting the exact
page geometry the extractor will rely on, which I haven't looked at yet: the
band rectangles and the header region.

Ran 2 shell commands (ctrl+o to expand)

● The geometry is simpler than expected—each 12pt band corresponds to one row,
with band rectangles directly giving column boundaries for label, prior, and
current. I'll merge the rule fragments, check on the install status, and then
build. · summary

Ran 1 shell command (ctrl+o to expand)

● Background command "Install packages; inspect band rects, rules, header words"
completed (exit code 0)

● Packages are installed (openpyxl, pytest, ruff, mypy). I'll write the package
now, starting with the data models and the PDF-reading layer.

● Write(pyproject.toml)
  ⎿ Wrote 33 lines to pyproject.toml
     1 [build-system]
     2 requires = ["setuptools>=68"]
     3 build-backend = "setuptools.build_meta"
     4
     5 [project]
     7 version = "0.1.0"
     8 description = "Rule-based parser for Japanese financial-statemen
       t tables in born-digital PDFs"
     9 requires-python = ">=3.10"
    10 dependencies = ["pymupdf", "openpyxl"]
    … +24 lines (ctrl+o to expand)

● Write(src¥fin_table_parser¥models.py) ⎿ Wrote 121 lines to src¥fin_table_parser¥models.py
     1 """Plain data containers shared by every stage of the pipeline.
     2
     3 Every stage takes and returns these dataclasses, so each stage ca
       n be unit-tested without a PDF.
     4 """
     5
     6 from __future__ import annotations
     7
     8 from dataclasses import dataclass, field
     9 from typing import Literal
    … +119 lines (ctrl+o to expand)

● Write(src¥fin_table_parser¥pdf_io.py)
  ⎿ Wrote 44 lines to src¥fin_table_parser¥pdf_io.py
     1 """PDF reading. This is the only module that imports PyMuPDF.
     2
     3 Swapping the PDF library (for example to pdfplumber, MIT license
       d) means rewriting only this file.
     4 """
     6 from __future__ import annotations
     7
     8 from pathlib import Path
     9
    10 import pymupdf
    … +39 lines (ctrl+o to expand)

● Write(src¥fin_table_parser¥normalize.py)
  ⎿ Wrote 64 lines to src¥fin_table_parser¥normalize.py
     1 """Text and number normalisation for Japanese financial statemen
       ts.
     2
     4 * ``△`` / ``▲`` prefix means negative (``△2,979`` -> -2979).
     5 * ``ー`` (a lone dash) means *nil*, which is different from 0.
     6 * ``△0`` is kept as 0 with ``negative_zero=True`` so the printed
        form is not lost.
     8 """
     9
    10 from __future__ import annotations
    … +59 lines (ctrl+o to expand)

● Write(src¥fin_table_parser¥labels.py)
  ⎿ Wrote 34 lines to src¥fin_table_parser¥labels.py
     1 """Label conventions of Japanese financial statements (the only
       account-name knowledge we use)."""
     2
     3 from __future__ import annotations
     4
     6
     7 from .models import RowType
     8 from .normalize import nfkc
     9
    10 _TOTAL = re.compile(r"(合計|総計)$")
    … +25 lines (ctrl+o to expand)

● Write(src¥fin_table_parser¥hierarchy.py)
  ⎿ Wrote 47 lines to src¥fin_table_parser¥hierarchy.py
     1 """Indentation levels and parent paths.
     2
     3 In these statements the indent of the label encodes the account
     4 the indent cluster on the page, never an absolute x position, be
       cause the first indent differs
     5 between pages (e.g. 62 pt on one page, 75 pt on another).
     6 """
     8 from __future__ import annotations
     9
    10 from collections.abc import Sequence
    … +45 lines (ctrl+o to expand)

● Now triage and the banded extractor, which is the core of the parser.

● Write(src¥fin_table_parser¥triage.py)
  ⎿ Wrote 45 lines to src¥fin_table_parser¥triage.py
     1 """Page triage: is the text layer usable, and which extraction st
       rategy applies?
     2
       ls triage raises
     4 ``UnsupportedPage`` with the reason; this is the hook where the O
       CR / Docling fallback attaches later.
     5 """
     7 from __future__ import annotations
     8
     9 import re
    … +48 lines (ctrl+o to expand)

● Write(src¥fin_table_parser¥strategies¥__init__.py)
  ⎿ Wrote 1 line to src¥fin_table_parser¥strategies¥__init__.py
     1 """Extraction strategies. Phase 1: ``banded`` (statements with al
       ternating row bands)."""

● Write(src¥fin_table_parser¥strategies¥banded.py)
  ⎿ Wrote 236 lines to src¥fin_table_parser¥strategies¥banded.py
     1 """Banded-statement strategy.
     3 The page draws one 12 pt coloured band per table row, and each b
       and is split into one rectangle
     4 per column. So the geometry gives us the rows *and* the column b
     5 positions are only used to decide which band / column a word bel
       ongs to.
     7 Pipeline: bands -> columns -> header -> rows -> rules/underlines
        -> hierarchy -> row types.
     8 """
    10 from __future__ import annotations
    … +265 lines (ctrl+o to expand)

● Write(src¥fin_table_parser¥pipeline.py)
  ⎿ Wrote 17 lines to src¥fin_table_parser¥pipeline.py
     1 """High-level entry points: PDF page -> Table (-> workbook)."""
     2
     3 from __future__ import annotations
     4
     5 from pathlib import Path
     6
     7 from .models import Table
     8 from .pdf_io import load_page
     9 from .strategies.banded import extract_banded
    10 from .triage import check_banded

… +8 lines (ctrl+o to expand)

● Let me run the extractor on the page and read the output before building anything on top of it.

Ran 1 shell command (ctrl+o to expand)

● The output looks correct—headers, units, periods, 訂正後 marker, and values all check out—but there are 16 phantom rows inflating the band count (60 vs ~44 real rows), which throws off indent levels. I'll investigate those extra bands before filtering them out. · summary

Ran 1 shell command (ctrl+o to expand)

● I found that phantom bands are thin 0.7pt empty slivers versus 12pt real rows, so I'll filter out bands significantly thinner than the typical height—a data-driven rule instead of a fixed threshold. · summary

● Update(src¥fin_table_parser¥strategies¥banded.py)
└ Added 16 lines, removed 5 lines

```
47
48
49  def find_bands(fills: Sequence[Rect]) -> list[tuple[float, floa
    t]]:
50 -    """Distinct (y0, y1) row bands, top to bottom."""
50 +    """Distinct (y0, y1) row bands, top to bottom.
51 +
52 +    The source also paints sub-pixel slivers (0.7 pt) behind th
   +e table rules. They hold no text,
53 +    so bands much thinner than the typical band are dropped.
54 +    """
55      ys = sorted({(round(r.y0, 1), round(r.y1, 1)) for r in fill
    s})
56 +    if not ys:
57 +        return []
58 +    heights = sorted(y1 - y0 for y0, y1 in ys)
59 +    typical = heights[len(heights) // 2]
60      bands: list[tuple[float, float]] = []
61      for y0, y1 in ys:
62 +        if y1 - y0 < MIN_BAND_RATIO * typical:
63 +            continue
64          if bands and abs(bands[-1][0] - y0) < 0.3:
65              continue
66          bands.append((y0, y1))
...
68
69
70  def find_columns(fills: Sequence[Rect]) -> list[Interval]:
61 -    """Column x-intervals: the most common set of rectangle x-e
   -xtents across bands."""
71 +    """Column x-intervals: the most common set of rectangle x-e
   +xtents across row bands."""
72 +    band_keys = set(find_bands(fills))
73      per_band: dict[tuple[float, float], set[Interval]] = {}
74      for r in fills:
64 -        per_band.setdefault((round(r.y0, 1), round(r.y1, 1)), s
   -et()).add(
65 -            (round(r.x0, 1), round(r.x1, 1))
66 -        )
75 +        key = (round(r.y0, 1), round(r.y1, 1))
76 +        if key in band_keys:
77 +            per_band.setdefault(key, set()).add((round(r.x0, 1)
   +, round(r.x1, 1)))
78      most_common = Counter(tuple(sorted(s)) for s in per_band.va
    lues()).most_common(1)
79      return list(most_common[0][0]) if most_common else []
80
```

● Update(src¥fin_table_parser¥strategies¥banded.py)
└ Added 1 line

```
22  FULL_RULE_RATIO = 0.9  # a rule at least this wide as the colum
    n is a "full" rule
23  THIN_RULE_MAX_HEIGHT = 1.5
24  HEAVY_RULE_MIN_HEIGHT = 1.5
25 +MIN_BAND_RATIO = 0.5  # bands thinner than this fraction of the
   + typical band are not rows
26
27  Interval = tuple[float, float]
28
```

● Update(src¥fin_table_parser¥strategies¥banded.py)
└ Added 4 lines, removed 1 line

```
140      thin_rules = [r for r in rules if r.height < THIN_RULE_MAX
     _HEIGHT and r.y0 >= table_top - 1]
141      rows = _rows(geom.words, bands, label_col, value_cols, thi
     n_rules)
142
143 -    levels = cluster_levels([r.label_x0 for r in rows])
143 +    # Indent levels come from labelled rows only; a row withou
   +t a label has no indent to read.
```

```
144 +    labelled = [r for r in rows if r.label]
145 +    rank = dict(zip((r.index for r in labelled), cluster_level
    +s([r.label_x0 for r in labelled]), strict=True))
146 +    levels = [rank.get(r.index, 0) for r in rows]
147      paths = build_paths([r.label for r in rows], levels)
148      for row, level, path in zip(rows, levels, paths, strict=Tr
     ue):
149          row.level, row.path = level, path

Ran 1 shell command (ctrl+o to expand)

● Row count and hierarchy levels now line up with the 43 tables found and the
  page image. Next I'll run the Excel export, the workbook reload, and the
  accounting validator. · summary

● Write(src¥fin_table_parser¥export.py)
  ⎿  Wrote 164 lines to src¥fin_table_parser¥export.py
     1 """Excel export: one workbook per PDF, one sheet per statement,
     2
     3 Sheet layout (header row is found by the reader by looking for t
       he ``Label`` cell, so a few
     5
     6     A Row | B Level | C Row type | D Label | E Path | numeric va
     7     underline-flag columns | Rule above
     8
     9 Nil cells (``—``) are left empty in the numeric columns; the ``
    10 printed so that nil, ``0`` and ``△0`` stay distinguishable.
    … +176 lines (ctrl+o to expand)

● Write(src¥fin_table_parser¥xlsx_reader.py)
  ⎿  Wrote 82 lines to src¥fin_table_parser¥xlsx_reader.py
     1 """Read a workbook written by ``export`` back into ``Table`` obje
       cts.
     2
        they also catch export bugs and
     4 hand edits. Only the sheet layout is assumed, not the parser that
        produced it.
     5 """
     7 from __future__ import annotations
     8
     9 from pathlib import Path
    … +90 lines (ctrl+o to expand)

● Let me tidy that reader before moving on (the placeholder float in the tuple
  is dead weight), then write the validator.

● Update(src¥fin_table_parser¥xlsx_reader.py)
  ⎿  Added 33 lines, removed 13 lines
     42
     43      labels: list[str] = []
     44      levels: list[int] = []
     45 -    drafts: list[tuple[RowType, list[Value | None], list[bool],
        - bool, float]] = []
     45 +    drafts: list[tuple[RowType, list[Value | None], list[bool],
        + bool]] = []
     46      for cells in ws.iter_rows(min_row=HEADER_ROW + 1, values_on
         ly=True):
     47 -        if cells[LABEL_COL - 1] is None and all(c is None for c
        - in cells):
     47 +        if all(c is None for c in cells):
     48              continue
     49          row_type = str(cells[2])
     50          if row_type not in _ROW_TYPES:
     51              raise ValueError(f"sheet {ws.title!r}: bad row type
         {row_type!r}")
     52          values: list[Value | None] = []
     53          for j in range(n_cols):
     54 -            number = cells[num_start + j]
     55 -            raw = cells[raw_start + j]
     54 +            number, raw = cells[num_start + j], cells[raw_start
        + + j]
     55              parsed = parse_value(str(raw)) if raw is not None e
         lse None
     57 -            if number is not None:
     58 -                kind = "number"
     59 -                neg_zero = bool(parsed and parsed.negative_zero
        -)
     60 -                values.append(Value(raw=str(raw) if raw is not
        -None else str(number), number=int(number), kind=kind, negative_
        -zero=neg_zero))
     56 +            if number is None:
     57 +                values.append(parsed)  # nil / text / empty: de
        +cided by the raw text alone
     58              else:
     62 -                values.append(parsed)
     63 -        underlined = [cells[und_start + j] == "yes" for j in ra
        -nge(n_cols)]
     59 +                # The numeric cell is authoritative for the num
        +ber; raw text keeps the printed form.
     60 +                values.append(
```

```
    61 +                    Value(
    62 +                        raw=str(raw) if raw is not None else st
       +r(number),
    63 +                        number=int(number),
    64 +                        kind="number",
    65 +                        negative_zero=bool(parsed and parsed.ne
       +gative_zero),
    66 +                    )
    67 +                )
    68          labels.append(str(cells[LABEL_COL - 1] or ""))
    69          levels.append(int(cells[1] or 0))
    66 -        drafts.append((row_type, values, underlined, cells[rule
       -_col] == "yes", 0.0))  # type: ignore[arg-type]
    70 +        drafts.append(
    71 +            (
    72 +                cast(RowType, row_type),
    73 +                values,
    74 +                [cells[und_start + j] == "yes" for j in range(n
       +_cols)],
    75 +                cells[rule_col] == "yes",
    76 +            )
    77 +        )
    78
    79      paths = build_paths(labels, levels)
    80      rows = [
    70 -        Row(index=i, label=labels[i], level=levels[i], path=pat
       -hs[i], values=d[1], row_type=d[0], rule_above=d[3], underlined=
       -d[2])
    71 -        for i, d in enumerate(drafts)
    81 +        Row(
    82 +            index=i,
    83 +            label=labels[i],
    84 +            level=levels[i],
    85 +            path=paths[i],
    86 +            values=values,
    87 +            row_type=row_type,
    88 +            rule_above=rule_above,
    89 +            underlined=underlined,
    90 +        )
    91 +        for i, (row_type, values, underlined, rule_above) in en
       +umerate(drafts)
    92      ]
    93      meta_revision = ws["B4"].value
    94      return Table(
```

● Update(src¥fin_table_parser¥xlsx_reader.py)
  ⎿ Added 1 line

```
     7  from __future__ import annotations
     8
     9  from pathlib import Path
    10 +from typing import cast
    11
    12  from openpyxl import load_workbook
    13  from openpyxl.worksheet.worksheet import Worksheet
```

● Write(src¥fin_table_parser¥validate.py)
  ⎿ Wrote 221 lines to src¥fin_table_parser¥validate.py

```
     1 """Accounting-identity checks on parsed tables.
     3 These checks use only the numbers in the table, never the PDF, s
       o they are an independent signal
       me from the parser directly or
     5 from an Excel workbook (``xlsx_reader``), so the same checks val
     6
     7 Checks
     9 ``numeric_matches_raw``  the numeric cell equals the printed tex
       t (catches export bugs / edits)
       s components (rounding tolerated)
   … +263 lines (ctrl+o to expand)
```

● Let me run the validator on the parsed table and look at the actual numbers before writing the evaluation code.

  Ran 1 shell command (ctrl+o to expand)

● I confirmed the three off-by-1 net-value discrepancies aren't parser errors—they're expected rounding artifacts from independently rounded source figures, verified against the PDF text layer. I'll apply the same ±1-per-figure tolerance used for totals to the net check instead of requiring exact matches. · summary

● Update(src¥fin_table_parser¥validate.py)
  ⎿ Added 4 lines, removed 1 line

```
    173              if not _has_value(net, col):
    174                  continue
    175              expected = _number(gross, col) + sum(_number(rows[
         k], col) for k in deductions)
    176 -            findings.append(_compare("gross_less_accum_equals_
        -net", table, header, net, col, expected, tolerance=0,
    176 +            # Gross and accumulated depreciation are each roun
        +ded on their own, and the net is
    177 +            # rounded from the unrounded difference, so +-1 pe
```

```
    +r figure is normal (seen on this page).
178 +            n_terms = 1 + len(deductions)
179 +            findings.append(_compare("gross_less_accum_equals_
    +net", table, header, net, col, expected, tolerance=n_terms,
180                                          detail=f"{gross.label} +
     " + ", ".join(rows[k].label for k in deductions)))
181      return findings
182
```

● Update(src¥fin_table_parser¥validate.py)
  ⎿ Added 1 line, removed 1 line

```
 8  ------
 9  ``numeric_matches_raw``  the numeric cell equals the printed te
    xt (catches export bugs / edits)
10  ``total_equals_components``  every 合計 row equals the sum of i
    ts components (rounding tolerated)
11 -``gross_less_accum_equals_net``  gross + accumulated depreciati
   -on = net, exact
11 +``gross_less_accum_equals_net``  gross + accumulated depreciati
   +on = net (rounding tolerated)
12  ``assets_equal_liabilities_equity``  資産合計 = 負債純資産合計
    across the statement's pages
13
14  How a total's components are found (from the indent tree; in th
    ese statements a 合計 row sits at
```

● Update(src¥fin_table_parser¥validate.py)
  ⎿ Added 7 lines, removed 3 lines

```
16  * a valued sibling contributes its value, or its （純額） child
    's value if it has one;
17  * a section header (no values) contributes its closing 合計 chi
    ld.
18
19 -Rounding: every line is rounded to the unit separately, so a to
   -tal can differ from the sum of
20 -its lines by up to the number of lines. Such differences are re
   -ported as ``PASS_ROUNDING``;
21 -anything larger is ``FAIL``. Exact checks use tolerance 0.
19 +Rounding: every printed figure is rounded to the unit separatel
   +y (error < 1 unit each), so a total
20 +can differ from the sum of its lines by up to the number of fig
   +ures involved. Such differences are
21 +reported as ``PASS_ROUNDING``; anything larger is ``FAIL``. Onl
   +y the balance identity is exact
22 +(both sides are printed totals of the same quantity).
23 +
24 +Limit: the tolerance also hides real errors no larger than the
   +tolerance (see the mutation test
25 +in ``evaluate``).
26  """
27
28  from __future__ import annotations
```

● I'll write the ground truth manually from the text dump and page image rather than the parser's output, to avoid grading the parser against itself, and I'll note its limitations clearly in the report. · summary

● Write(tests¥fixtures¥ground_truth¥balance_sheet_after_correction.json)
  ⎿ Wrote 56 lines to tests¥fixtures¥ground_truth¥balance_sheet_after_correction.json

```
 1 {
 3   "page_no": 0,
 4   "annotation_method": "Hand-transcribed by Claude from the PDF
   plain-text dump (values, reading order) and the rendered page im
   arser output. Not yet reviewed by a human.",
 5   "statement": "連結貸借対照表",
 7   "revision": "訂正後",
 8   "headers": ["前連結会計年度 (2025年3月31日)", "当連結会計年度
   (2026年3月31日)"],
10   "underlined_cells": [[35, 1], [36, 1]],
… +50 lines (ctrl+o to expand)
```

● Next, the evaluation module: the parser-versus-ground-truth metrics, a baseline (PyMuPDF's built-in find_tables), and a mutation test that measures how well the accounting checks catch errors.

● Update(src¥fin_table_parser¥pdf_io.py)
  ⎿ Added 7 lines

```
19      return (r, g, b)
20
21
22 +def baseline_find_tables(pdf_path: str | Path, page_no: int = 0
   +) -> list[list[str]]:
23 +    """Rows from PyMuPDF's built-in ``find_tables()`` (used onl
   +y as an evaluation baseline)."""
24 +    with pymupdf.open(str(pdf_path)) as doc:
25 +        tables = doc[page_no].find_tables().tables
26 +        return [[c or "" for c in row] for t in tables for row
   +in t.extract()]
27 +
```

```
    28 +
    29  def load_page(pdf_path: str | Path, page_no: int = 0) -> PageGe
       ometry:
    30      """Read words and filled rectangles of one page (0-based ``
       page_no``)."""
    31      with pymupdf.open(str(pdf_path)) as doc:
```

● Write(src¥fin_table_parser¥evaluate.py)
```
  ⎿  Wrote 296 lines to src¥fin_table_parser¥evaluate.py
     1 """Evaluation: parser output vs ground truth, a baseline, and a f
     2
     3 Three independent questions:
     4
       -made ground truth?
     6 2. ``score_baseline``       How does PyMuPDF's built-in ``find_ta
     7 3. ``mutation_test``        If a single cell were wrong, would th
       e accounting checks notice?
     8    (This measures the validator, not the parser: it answers "how
     9 """
    … +334 lines (ctrl+o to expand)
```

● That nil-vs-zero condition I wrote has a precedence tangle. Let me fix it cleanly before it quietly mis-scores something.

● Update(src¥fin_table_parser¥evaluate.py)
```
  ⎿  Added 3 lines, removed 2 lines
    183                  if t_raw is not None:
    184                      s.cells.fn += 1
    185              # Strictly typed comparison (number / nil / negati
         ve zero) incl. empty cells.
    186 -            ok = _value_signature(parse_value(t_raw) if t_raw
        -else None) == _value_signature(p_val)
    186 +            t_val = parse_value(t_raw) if t_raw else None
    187 +            ok = _value_signature(t_val) == _value_signature(p
        +_val)
    188              r = ratio("cell.parsed_value")
    189              r.total += 1
    190              r.correct += int(ok)
    190 -            if t_raw is not None and (parse_value(t_raw) or Va
        -lue("", None, "text")).kind == "nil" or t_raw in ("0", "△0"):
    191 +            if t_val is not None and (t_val.kind == "nil" or t
        +_val.number == 0):
    192                  rr = ratio("cell.nil_vs_zero")
    193                  rr.total += 1
    194                  rr.correct += int(ok)
```

● Write(src¥fin_table_parser¥report.py)
```
  ⎿  Wrote 78 lines to src¥fin_table_parser¥report.py
     1 """Render evaluation results as a Markdown report."""
     2
     3 from __future__ import annotations
     4
     5 from collections.abc import Sequence
     6
     7 from .evaluate import GroundTruth, MutationResult, PRF, Ratio, Sc
       ores
     8 from .models import Finding
     9 from .validate import summarize
    … +95 lines (ctrl+o to expand)
```

● Write(src¥fin_table_parser¥cli.py)
```
  ⎿  Wrote 92 lines to src¥fin_table_parser¥cli.py
     1 """Command line: parse a PDF page to Excel, check a workbook, or
        evaluate against ground truth.
     3     python -m fin_table_parser parse  in.pdf -o output/in.xlsx [
       --page 0]
     4     python -m fin_table_parser check  output/in.xlsx
       s/reports/metrics.md
     6 """
     7
     9
    10 import argparse
    … +97 lines (ctrl+o to expand)
```

● Write(src¥fin_table_parser¥__init__.py)
```
  ⎿  Wrote 7 lines to src¥fin_table_parser¥__init__.py
     1 """Rule-based parser for Japanese financial-statement tables in b
       orn-digital PDFs."""
     2
     3 from .models import Finding, Row, Table, Value
     4 from .pipeline import parse_page
     5 from .validate import validate_tables
     6
     7 __all__ = ["Finding", "Row", "Table", "Value", "parse_page", "val
       idate_tables"]
```

● Write(src¥fin_table_parser¥__main__.py)
```
  ⎿  Wrote 5 lines to src¥fin_table_parser¥__main__.py
     1 import sys
     2
```

```
    3 from .cli import main
    4
    5 sys.exit(main())
```

● I'll install the package in editable mode (my own project, no new third-party dependencies) and run the full end-to-end evaluation.

  Ran 1 shell command (ctrl+o to expand)

● The parser matches ground truth on this page, but since I developed against it, I won't read too much into that yet. Next I'll run two independent stress tests: the 訂正前/訂正後 diff (expecting exactly 3 cells) and the liabilities page, which tests the Assets = Liabilities + Equity check. · summary

  Ran 1 shell command (ctrl+o to expand)

● The original page doesn't align row-for-row with the corrected one, which could be a real parser failure on a slightly different page. Let me look at what it produced.

  Ran 1 shell command (ctrl+o to expand)

● I found a real issue on this variant page—duplicated rows from overlapping bands and an empty header where the heavy rule wasn't detected. I'll examine the page's geometry next. · summary

  Ran 1 shell command (ctrl+o to expand)

● The original page has no filled black rules—the total-row bands are 12.7pt and overlap neighboring rows, suggesting the lines are drawn as strokes rather than fills. I'll verify this next. · summary

  Ran 1 shell command (ctrl+o to expand)
● This variant renders rules as stroked lines rather than filled rectangles, and the 12 word-width underlines in the original filing confirm underlines aren't a correction marker. I'll update pdf_io to treat dark horizontal strokes as rules and have band detection drop overlapping duplicates. · summary

● Update(src¥fin_table_parser¥pdf_io.py)
  └ Added 11 lines, removed 5 lines
```
    34          fills: list[Rect] = []
    35          rules: list[Rect] = []
    36          for drawing in page.get_drawings():
    37 +            r = drawing["rect"]
    38              color = drawing.get("fill")
    38 -            if color is None:
    39 +            if color is not None:
    40 +                fill = _round_color(color)
    41 +                rect = Rect(r.x0, r.y0, r.x1, r.y1, fill)
    42 +                (rules if fill == _BLACK else fills).append(rec
       +t)
    43                  continue
    40 -            r = drawing["rect"]
    41 -            fill = _round_color(color)
    42 -            rect = Rect(r.x0, r.y0, r.x1, r.y1, fill)
    43 -            (rules if fill == _BLACK else fills).append(rect)
    44 +            stroke = drawing.get("color")
    45 +            # Some producers draw table rules as stroked horizo
       +ntal lines instead of thin filled
    46 +            # rectangles; normalise them to the same Rect form
       +(centred on the line).
    47 +            if stroke is not None and r.height < 0.01 and sum(s
       +troke[:3]) < 0.3:
    48 +                half = (drawing.get("width") or 0.5) / 2
    49 +                rules.append(Rect(r.x0, r.y0 - half, r.x1, r.y0
       + + half, _BLACK))
    50          return PageGeometry(
    51              page_no=page_no,
    52              width=page.rect.width,
```

● Update(src¥fin_table_parser¥strategies¥banded.py)
  └ Added 3 lines, removed 1 line
```
    62      for y0, y1 in ys:
    63          if y1 - y0 < MIN_BAND_RATIO * typical:
    64              continue
    65 -        if bands and abs(bands[-1][0] - y0) < 0.3:
    65 +        # Some pages paint a total row's band a little taller t
       +han its neighbours, so two
    66 +        # bands cover the same row. Keep the first and drop a b
       +and that mostly overlaps it.
    67 +        if bands and y0 < bands[-1][1] - MAX_BAND_OVERLAP * (y1
       + - y0):
    68              continue
    69          bands.append((y0, y1))
    70      return bands
```

● Update(src¥fin_table_parser¥strategies¥banded.py)
  └ Added 1 line
```
    23  THIN_RULE_MAX_HEIGHT = 1.5
    24  HEAVY_RULE_MIN_HEIGHT = 1.5
```

```
    25  MIN_BAND_RATIO = 0.5  # bands thinner than this fraction of the
         typical band are not rows
    26 +MAX_BAND_OVERLAP = 0.5  # a band overlapping the previous one b
       +y more than this fraction is a duplicate
    27
    28  Interval = tuple[float, float]
    29
```

Ran 1 shell command (ctrl+o to expand)

● Both stress tests pass — the diff test catches exactly the 3 expected corrected cells, and the balance identity holds on real data for both years. One remaining defect: page 9 lacks a section title, causing the statement name to pick up the page header, so I'll restrict acceptance to lines that actually name a statement. · summary

● Update(src¥fin_table_parser¥strategies¥banded.py)
  └ Added 4 lines, removed 1 line

```
    188              unit = m.group(1).strip()
    189          else:
    190              candidates.append(line)
    191 -    statement = strip_section_number(candidates[-1]) if candid
        -ates else ""
    191 +    # Only a line that names a statement counts; a continuatio
        +n page has no title, and its
    192 +    # last line above the table is just the running page heade
        +r.
    193 +    named = [c for c in candidates if any(k in nfkc(c) for k i
        +n STATEMENT_KEYWORDS)]
    194 +    statement = strip_section_number(named[-1]) if named else
        +""
    195      return unit, statement
    196
    197
```

● Update(src¥fin_table_parser¥strategies¥banded.py)
  └ Added 1 line, removed 1 line

```
    14  from collections.abc import Iterable, Sequence
    15
    16  from ..hierarchy import build_paths, cluster_levels
    17 -from ..labels import classify_row
    17 +from ..labels import STATEMENT_KEYWORDS, classify_row
    18  from ..models import PageGeometry, Rect, Row, Table, Value, Wor
        d
    19  from ..normalize import nfkc, parse_value, period_end, strip_se
        ction_number
    20
```

● Update(src¥fin_table_parser¥labels.py)
  └ Added 9 lines

```
    10  _TOTAL = re.compile(r"(合計|総計)$")
    11  _NET = re.compile(r"[ ((]純額[) )]")
    12
    13 +# A title line must contain one of these to be taken as the sta
       +tement name.
    14 +STATEMENT_KEYWORDS = (
    15 +    "貸借対照表",
    16 +    "損益計算書",
    17 +    "包括利益計算書",
    18 +    "キャッシュ・フロー計算書",
    19 +    "株主資本等変動計算書",
    20 +)
    21 +
    22  ASSETS_TOTAL = "資産合計"
    23  LIABILITIES_EQUITY_TOTALS = {
    24      "負債純資産合計",
```

Ran 1 shell command (ctrl+o to expand)

● Now the tests. Besides the usual unit tests, I'm including the end-to-end workbook checker test you asked about: corrupt a cell in the Excel file and confirm the checker flags it.

Ran 2 shell commands (ctrl+o to expand)

● The shell choked on the heredoc quoting. I'll use the file-writing tool instead.

● Write(tests¥unit¥test_validate.py)
  └ Wrote 87 lines to tests¥unit¥test_validate.py

```
     1 """Validator tests on small synthetic tables (no PDF needed)."""
     2
     3 from fin_table_parser.labels import classify_row
     5 from fin_table_parser.normalize import parse_value
     6 from fin_table_parser.validate import check_balance_identity, su
       mmarize, validate_tables
     7
     9 def make_table(spec, statement="BS"):
    10     """spec: list of (level, label, raw values...) with raw=None
        for an empty cell."""
```

```
    … +91 lines (ctrl+o to expand)

● Write(tests¥integration¥test_balance_sheet_page.py)
  └ Wrote 117 lines to tests¥integration¥test_balance_sheet_page.py
     1 """End-to-end tests on the real sample pages."""
     2
     3 import pytest
     4 from openpyxl import load_workbook
     5
        score_against_truth
     7 from fin_table_parser.export import HEADER_ROW, write_workbook
     8 from fin_table_parser.pipeline import parse_page
     9 from fin_table_parser.triage import UnsupportedPage
    10 from fin_table_parser.validate import summarize, validate_tables
    … +127 lines (ctrl+o to expand)

  Ran 1 shell command (ctrl+o to expand)

● 47 tests pass. Now static analysis (AGENT.md asks for lint and type checking).

  Ran 1 shell command (ctrl+o to expand)

● Mostly long lines. I'll let ruff format wrap them, then fix the rest by hand.

  Ran 1 shell command (ctrl+o to expand)

● The formatting changes are just from running ruff format, and remaining
  line-length warnings are long strings the formatter can't wrap, so I'll leave
  those alone and instead fix the actual issues: the unused loop variable and
  the mypy error. · summary

  Ran 1 shell command (ctrl+o to expand)

● Update(src¥fin_table_parser¥validate.py)
  └ Added 1 line, removed 1 line
    142              continue
    143          siblings = [s for s in _children(rows, parents, parent
         s[t_idx]) if s < t_idx]
    144          if any(rows[s].row_type == "total" for s in siblings):
    145 -            for col, header in enumerate(table.headers):
    145 +            for header in table.headers:
    146                  findings.append(
    147                      Finding(
    148                          "total_equals_components",

● Update(src¥fin_table_parser¥export.py)
  └ Added 1 line, removed 1 line
    49
    50  def write_workbook(tables: Sequence[Table], findings: Sequence[
        Finding], path: str | Path) -> Path:
    51      wb = Workbook()
    52 -    wb.remove(wb.active)  # type: ignore[arg-type]
    52 +    wb.remove(wb.active)
    53      names: list[str] = []
    54      for table in tables:
    55          name = sheet_name(table.statement, names)

  Ran 1 shell command (ctrl+o to expand)

● Lint, types and tests are clean. Now the real CLI flow on the actual page:
  parse to Excel, then run the standalone checker on the Excel file, then
  spot-check the workbook contents.

  Ran 1 shell command (ctrl+o to expand)

● Everything works end to end. Now the documentation (AGENT.md requires modular
  docs: concepts, details, and a work log kept separately).

● Write(docs¥explanations¥01-how-the-parser-works.md)
  └ Wrote 109 lines to docs¥explanations¥01-how-the-parser-works.md
     1 # How the parser works (concepts)
     3 Audience: a developer new to the project. Code: [src/fin_table_p
       arser/](../../src/fin_table_parser/).
     5 ## The one-paragraph version
     6
       coloured 12 pt **band** per table row.
     8 We read the words and the drawn rectangles with PyMuPDF, use the
     9 rectangles as columns, put each word into the cell it falls in,
       then add the financial-statement
        = nil) and export an Excel file.
    … +145 lines (ctrl+o to expand)
● Write(docs¥explanations¥02-accounting-checks.md)
  └ Wrote 63 lines to docs¥explanations¥02-accounting-checks.md
     1 # Accounting checks (how we know whether the parse is right)
     3 Code: [validate.py](../../src/fin_table_parser/validate.py). Tes
       ts: `tests/unit/test_validate.py`.
     4
     6
     7 A balance sheet is self-checking. If a digit is mis-read, a tota
     8 only the numbers in the table, never the PDF, so they are an **i
```

```
      ndependent** signal about parser errors.
    9
       **Excel file** (`xlsx_reader`), so the
  … +83 lines (ctrl+o to expand)

● Write(docs¥worklog¥phase1-worklog.md)
  ⎿  Wrote 50 lines to docs¥worklog¥phase1-worklog.md
     1 # Phase 1 work log
     2
       ll as possible; Excel output, one workbook
     4 per PDF, one sheet per statement; evaluate with metrics.
     5
     7
     8 `src/fin_table_parser/`: models, pdf_io, triage, strategies/band
       ed, hierarchy, normalize, labels,
       Tests in `tests/unit` and `tests/integration`
    10 (47 tests). Ground truth in `tests/fixtures/ground_truth/`.
  … +70 lines (ctrl+o to expand)

● Update(docs¥plans¥phase1-implementation-plan.md)
  ⎿  Added 3 lines, removed 1 line
     1  # Phase 1 Implementation Plan: Financial-Statement Table Parser
     2
     3 -> Status: **PROPOSED, awaiting approval.** No code has been writ
       -ten. (AGENT.md: plan first.)
     3 +> Status: **APPROVED with a narrower scope** (user decision): Ph
       +ase 1 targets `balance_sheet_after_correction.pdf`
     4 +> only, Excel output, one workbook per PDF. Implemented; see [..
       +/worklog/phase1-worklog.md](../worklog/phase1-worklog.md).
     5 +> The text below is the original proposal and still describes Cl
       +asses B/C and later phases as future work.
     6  > Test corpus: `complete_pdf.pdf` (19 pages, original) + `balanc
       e_sheet_after_correction.pdf` (1 page, corrected).
     7  > Background: [../research/01-table-parser-landscape.md](../rese
       arch/01-table-parser-landscape.md)
     8

● Advising using Opus 5.5

● The deliverable is on disk and verified, so this is the point to get an
  outside read before I report.

● Advising using Opus 5.5
  ⎿  √ Advisor has reviewed the conversation and will apply the feedback

● Good catches. I'll fix the baseline slice first, since it changes a published
  number.

  Ran 1 shell command (ctrl+o to expand)

● Update(tests¥fixtures¥ground_truth¥balance_sheet_after_correction.json)
  ⎿  Added 1 line, removed 1 line
     1  {
     2    "source_pdf": "balance_sheet_after_correction.pdf",
     3    "page_no": 0,
     4 -  "annotation_method": "Hand-transcribed by Claude from the PDF
       -plain-text dump (values, reading order) and the rendered page im
       -age (indent level, row type, underlines). NOT derived from the p
       -arser output. Not yet reviewed by a human.",
     4 +  "annotation_method": "Written by Claude: values from the PDF p
       +lain-text dump (the same text layer the parser reads), indent le
       +vel / row type / underlines from the rendered page image. Writte
       +n after the parser's rows had been displayed, so it is not blind
       + to the parser output. Values were not re-read from the pixels.
       +Not yet reviewed by a human.",
     5    "statement": "連結貸借対照表",
     6    "unit": "千円",
     7    "revision": "訂正後",

● Update(src¥fin_table_parser¥report.py)
  ⎿  Added 6 lines, removed 1 line
      98      lines.append(f"| **All** | **{mutation.total}** |")
      99      lines.append("")
     100      if mutation.never_detected:
     101 -        lines += ["Cells where *no* injected error was ever ca
         -ught (not covered by any check):", ""]
     101 +        lines += [
     102 +            "Cells where *no* injected error was ever caught:
         +either no check covers the cell, or "
     103 +            "every injected change stayed within the rounding
         +tolerance (a value of 0, for "
     104 +            "example, can only change by +-1):",
     105 +            "",
     106 +        ]
     107          lines += [f"- {c}" for c in mutation.never_detected]
     108          lines.append("")
     109      return "¥n".join(lines)

● Update(docs¥explanations¥02-accounting-checks.md)
  ⎿  Added 2 lines, removed 1 line
```

```
   46  checks whether a new FAIL appears. On the sample page: errors i
       n the middle or leading digit, transposed
   47  digits, sign flips and dropped digits are caught 97–100% of the
        time, but **a ±1 error in the last digit is
   48  caught only 14% of the time**, because it hides inside the roun
       ding tolerance. One cell (貸倒引当金, `△0`)
   49 -is not caught for any error. So the checks are strong against g
      -ross mistakes and weak against tiny ones.
   49 +is not caught for any error: it is covered by the 流動資産合計
      +check, but its value is 0, so every
   50 +possible change is ±1 and falls inside the tolerance. So the c
      +hecks are strong against gross mistakes and weak against tiny o
      +nes.
   51  Details: [docs/reports/phase1-metrics.md](../reports/phase1-met
       rics.md).
   52
   53  ## Using it across pages
```

Ran 1 shell command (ctrl+o to expand)

● The baseline now scores 43/43 on labels, so it ties on text and everything this parser adds is structure. I'll update the work log to match.

● Update(docs¥worklog¥phase1-worklog.md)
  └ Added 5 lines, removed 1 line

```
   29  ## Results (details: docs/reports/phase1-metrics.md)
   30
   31  - Ground truth: 43/43 rows, 86/86 cells, hierarchy, row types,
       headers, unit, period dates all correct; page exact match.
   32 -- Baseline `find_tables()`: cell text equally good, but no head
      -er, hierarchy, row types or unit.
   32 +- Baseline `find_tables()`: ties on labels (43/43) and cell tex
      +t, but produces no header, period dates, hierarchy,
   33 +  row types, unit or flags. Everything this parser adds is stru
      +cture. (An earlier report showed the baseline at
   34 +  42/43 because the evaluation wrongly dropped its first row; f
      +ixed.)
   35 +- Ground-truth caveat: written by Claude after the parser's row
      +s were on screen, values from the same text layer;
   36 +  not blind to the parser and not re-read from the pixels.
   37  - Accounting checks on the page: 0 FAIL (8 PASS, 13 PASS_ROUNDI
       NG, 1 SKIP: balance identity needs the other page).
   38  - Original vs corrected page: differ in exactly the 3 corrected
        cells.
   39  - Assets = liabilities + net assets verified on the original fi
       ling's pages 8 + 9.
```

● I built the parser for balance_sheet_after_correction.pdf. It produces output/balance_sheet_after_correction.xlsx (one sheet for the statement, plus Validation and Metadata sheets). 47 tests pass, and ruff and mypy (strict) are clean.

How it works

The PDF has an exact text layer and draws one coloured 12 pt band per row. The parser copies each word into the band and column it falls in, so it can't invent digits. Indentation gives the account hierarchy. △ becomes negative, ― stays nil rather than zero, and △0 is kept as printed.

2,002,429 → 2002429　　△2,979 → -2979　　― → nil (empty cell, raw "―")　　△0 → 0 (flagged)
その他 → 資産の部 > 固定資産 > 有形固定資産 > その他　(the 4 その他 rows each get their own path)

Results

The first two results don't depend on my ground truth, so I'd weight them most.

Test	Result
Original vs corrected page	Differ in exactly the 3 corrected cells: 貸倒引当金 0→△0, 投資有価証券 53,925→218,836, 関係会社株式 166,033→1,123
Assets = Liabilities + Equity, on the original filing's p. 8 + p. 9	Holds: 7,534,110 = 7,534,110 (prior year) and 5,135,249 = 5,135,249 (current year)
Accounting checks on this page	0 FAIL: 8 exact, 13 within rounding. The balance identity is skipped because this page only has the assets half

Vs my ground truth (43 rows, 86 cells)	All correct: labels, indent levels, parent paths, row types, values, headers, period dates, unit, 訂正後 marker
PyMuPDF find_tables() baseline	Ties on labels and cell text (43/43). It produces no header, hierarchy, row types or unit

Caveats on the ground-truth result:
- Ground truth written by me: I wrote it from the text dump and the page image, after the parser's rows were on screen. The values come from the same text layer the parser reads and were not re-read from the pixels. Please review tests/fixtures/ground_truth/balance_sheet_after_correction.json yourself.
- One page only: I developed against it, so 100% here is an upper bound, not what to expect on new pages.

The tally idea

It's built, and the logic is in validate.py. python -m fin_table_parser check file.xlsx reads the Excel file and runs the checks. I confirmed it flags a deliberately edited cell: changing one digit of 売掛金 in the sheet makes 流動資産合計 fail. It has three limits:
- Identity only on full statements: Assets = Liabilities + Equity needs both pages in one workbook. On this one-page PDF it reports SKIP.
- It says what, not where: the identity shows that something is wrong. The subtotal and net-value checks show where.
- Small errors slip through: each printed figure is rounded separately, so totals legitimately differ from their parts by a few units. Exact matching raised false alarms on correct data (1,880,082 − 1,640,253 = 239,829, but the page prints 239,828). I injected errors one cell at a time to see what the checks catch. Wrong middle or leading digits, swapped digits, sign flips and dropped digits are caught 97-100% of the time. A +1 error in the last digit is caught only 14% of the time, because it hides inside the rounding tolerance.

Cloud parsers (AWS Textract, Azure, Google)

I didn't say they're really good. The evidence I found was a vendor-run benchmark with no Japanese results. On a text-layer PDF they re-read pixels, so at best they tie what we already get exactly. They also cost money per page, and your documents leave your machine.

I'd skip them for now. They're worth considering only as a fallback behind the scanned-page hook, or as an independent second reader when evaluating.

Things you should know

- Packages installed: openpyxl (Excel), pytest, ruff and mypy. I also installed the project itself in editable mode (pip install -e .), which you hadn't explicitly approved. Say if you want that removed.
- Beyond the agreed scope: I tested on the sibling pages in complete_pdf.pdf. That exposed two real bugs: it draws its rules as stroked lines and overlaps its row bands, which gave duplicated rows and an empty header. I fixed both. Some tests depend on that file and skip if it's missing.
- Underlines: 12 appear in the original filing where nothing was corrected, so they only mean "an underline exists", not "corrected". Corrections are found by diffing versions.
- Known limits:
  - Only banded statements are supported.
  - Grid-ruled pages, such as the equity statement, are not handled yet.
  - A label wrapped over two lines in one band is not handled yet.
- Not committed: this directory isn't a git repo, and nothing has been pushed.

The documentation is in docs/: how the parser works (docs/explanations/01-how-the-parser-works.md), the accounting checks (docs/explanations/02-accounting-checks.md), the work log (docs/worklog/phase1-worklog.md) and the metrics report (docs/reports/phase1-metrics.md).

Do you approve this milestone?

If so, I'd suggest one of these next:
1. You review the ground truth file.
2. I read the 74 numbers off the rendered image, as a check independent of the text layer.
3. We move to Phase 2, the grid-ruled pages.

> Before we move onto the phase 2 - confirm that u have not hardcoded the parser or anything like that. Like Coordinate-hardcoding.
If not then we are positive.
Before we discuss our next move, we need to understand our actual goal: Given a PDF page containing a Japanese consolidated balance sheet (it may have slight variation in layout), extract its table into structured data while preserving the row labels, values, columns, hierarchy, units, and accounting signs.

so hard coding is definetly not the option for us.
And again we cannot rule out scanned pdfs either so what we'll need would be a hybrid approach.
So considering the ultimate goal - what shall be our next move? improving the current algo for parser? working on the hybrid approach? what shall it be?

● I won't just assert that, so I'll audit the code and then test it. First, every numeric literal in the source, and then a direct experiment: re-scale and shift the page and see whether the output survives.

Ran 1 shell command (ctrl+o to expand)

● The audit turned up no hardcoded page coordinates or PDF-specific strings, but it did find tuned thresholds—several in absolute points—which could hurt robustness. I'll now test this directly by re-rendering the same page at different scales and offsets and comparing against ground truth. · summary

Ran 1 shell command (ctrl+o to expand)

● The parser handles shifts and scales from 0.7–2.0 well, but breaks at 0.5× and 3× because some thresholds are in absolute points rather than relative units. I'll fix this by making all geometric thresholds relative to the page's row height—let me check the current code. · summary

Read 1 file (ctrl+o to expand)

● I'll make every geometric tolerance a fraction of the page's own typical row height (12 pt on this page, but derived, never assumed).

Ran 1 shell command (ctrl+o to expand)

● The shell mangles that heredoc again. I'll write the edit script as a file and run it.

Made 1 scratchpad edit +217, ran 1 shell command (ctrl+o to expand)

● Updating the two unit tests for the new signature, and adding the scale-invariance test permanently so this can't regress.

Ran 1 shell command (ctrl+o to expand)

● Write(tests¥integration¥test_layout_invariance.py)
  ⎿ Wrote 35 lines to tests¥integration¥test_layout_invariance.py
     1 """The parser must not depend on absolute coordinates, page size
     2
     3 The sample page is re-rendered (vector content and text layer in
       tact) at different scales and
       y.
     5 """
     7 import pymupdf
     8 import pytest
     9
       truth
    … +34 lines (ctrl+o to expand)

Ran 1 shell command (ctrl+o to expand)

● I broke something in the refactor. Let me see the actual error.
Ran 1 shell command (ctrl+o to expand)

● A name collision: my new length variable unit is shadowed by the existing currency unit ("千円"). I'll rename the currency one.

Ran 1 shell command (ctrl+o to expand)

● All 54 tests pass across different scales. Now I'll verify the parser's dependence on colored row bands by stripping graphics from the page, since many real balance sheets and OCR output won't have them. · summary

Ran 3 shell commands (ctrl+o to expand)

● Confirmed, with the graphics gone all 132 words are intact but the parser (correctly) refuses rather than guessing. Let me regenerate the artifacts and update the docs with what this round found.

Ran 1 shell command (ctrl+o to expand)

● Update(docs¥worklog¥phase1-worklog.md)
  ⎿ Added 7 lines
     24 | 4 | Continuation page (p. 9) took the running page header as
         the statement name | stress test | a title line must contain a
         statement keyword |
     25 | 5 | Earlier claim that underlines mark corrections was wrong
         | original filing has 12 word-width underlines | underline is a
          low-trust flag only; corrections found by diffing versions |
     26
     27 +| 6 | Several tolerances were absolute points (rule height 1.5
        +pt, line gap 3 pt, underline offsets): headers broke when the s
        +ame page was rendered at 0.5x or 3x | scale/offset experiment (

```
      +user asked to confirm nothing is hard-coded) | every geometric
      +tolerance is now a fraction of the page's own median row height
      + (`row_height`, constants at top of `banded.py`); `tests/integr
      +ation/test_layout_invariance.py` re-renders the page at 0.4x-3x
      + with offsets and requires an exact match |
   28 +
   29 +Audit result for "is anything hard-coded": no page coordinates
      +and no PDF-specific strings anywhere in `src/`
   30 +(grep-verified). What *is* fixed: relative tolerances (now scal
      +e-free), the domain vocabulary in `labels.py`
   31 +(合計, 純額, 資産合計, 負債純資産合計, statement keywords), and
      + structural assumptions (rows are drawn as coloured
   32 +bands; a total sits at the same indent as its components).
   33 +
   34  Problems 3-4 were found only because of the stress tests; a sin
       gle-page score of 100% did not reveal them.
   35
   36  ## Results (details: docs/reports/phase1-metrics.md)

● Update(docs¥worklog¥phase1-worklog.md)
  └ Added 7 lines
   53  - One page, developed against: 100% here is an upper bound for
       unseen pages, not an expected score.
   54  - Static analysis: ruff and mypy (strict) clean; 47 tests pass.
   55
   56 +## Known gap found after Phase 1 (evidence, not yet fixed)
   57 +
   58 +With all graphics removed from the page (text layer untouched:
      +132 words), the parser raises `UnsupportedPage`.
   59 +It needs the coloured row bands. A balance sheet without bands,
      + or the output of an OCR engine (words with boxes,
   60 +no drawings), cannot be parsed yet, although labels, numbers, i
      +ndents and headers are all present in the words.
   61 +The band-stripped page is a ready test case: same ground truth,
      + no graphics.
   62 +
   63  ## Not done / next
   64
   65  - Phase 2: grid-ruled pages (equity statement, segment notes),
       multi-line labels, title inheritance across pages.

● Update(docs¥explanations¥01-how-the-parser-works.md)
  └ Added 7 lines
    88  **6. Rules can be filled rectangles or stroked lines**, depend
        ing on the PDF producer.
    89  `pdf_io.load_page` normalises both to the same `Rect` form.
    90
    91 +## No absolute coordinates
    92 +
    93 +Nothing in the code is a page coordinate. Every geometric tole
       +rance is a fraction of the page's own median
    94 +row height (`row_height` in `banded.py`), so the result does n
       +ot depend on page size, zoom or font size.
    95 +`tests/integration/test_layout_invariance.py` proves it by re-
       +rendering the sample page at 0.4x-3x with offsets.
    96 +What *is* assumed: the vocabulary in `labels.py`, and that row
       +s are drawn as coloured bands.
    97 +
    98  ## What happens on a page this parser cannot read
    99
   100  `triage.check_banded` raises `UnsupportedPage` with a reason (
        scanned page, garbled text layer, no row

  Ran 2 shell commands (ctrl+o to expand)

● No page coordinates are hardcoded, but you were right to ask. The audit and a
  re-scaling test found a milder version of the same problem, which I fixed. The
  next step I recommend is a band-free core that both the text layer and OCR
  can feed. I haven't started on it.

  Is anything hardcoded?

  - Page coordinates and PDF-specific strings: none in src/. I grepped for this
    page's band edges, column edges, file names and the company name, and found
    nothing.
  - Shift and scale test: I re-rendered the page at different scales and
    offsets. It was exact at shifts and at 0.7×-2×, but failed on the headers at
    0.5× and 3×.
  - What I fixed: I had written some tolerances in absolute points (rule height,
    line gap, underline offset). Every tolerance is now a fraction of the
    page's own median row height. A new test,
    tests/integration/test_layout_invariance.py, re-renders the page at 0.4×-3×
    and requires an exact match. All 54 tests pass, and ruff and mypy are clean.
  - What is still assumed, not hardcoded:
    - The vocabulary in labels.py (合計, 純額, 資産合計, 負債純資産合計). It
      needs extending for IFRS wording.
    - That a total sits at the same indent as its components.
    - That rows are drawn as coloured bands. This is the real weakness.

  The gap that matters most
```

I removed all graphics from the page, leaving the 132 words untouched. The parser refused with UnsupportedPage, which is right, since it needs the bands. That page is a realistic stand-in for a balance sheet without shaded rows. OCR output has the same shape: words with boxes and no drawings.

All the information is still in the words: labels, numbers, indents and headers. Rows come from clustering by y, columns from the right edges of numbers, and hierarchy from indent. The bands were a shortcut for those steps.

Recommended next step

Do both, in this order. They share one piece of work:

1. Build a band-free core. The input is words with boxes, and drawings are only hints. This helps with layout variation and is the core of the hybrid, because an OCR engine produces exactly this input.
   - Test it for free: the band-stripped page has the same ground truth. The existing band-based result also serves as a reference on every banded page.
2. Add the hybrid on top. Triage already decides when a page needs a fallback. Only the source of words changes: the text layer for digital PDFs, or a rasterise-then-OCR step for scans. The same core and the same accounting checks then run on both.
3. Later, a vision-model second reader. It would run only where OCR is uncertain or a check fails.

Improving the current algorithm first would polish a path that scans can't use. Building the hybrid first would mean writing the core twice.

Questions for you before I write the plan

- OCR engine: I'd compare two engines for Japanese on a rasterised copy of this page. Both need new packages and models, so I'll ask before installing anything. Is a larger download acceptable?
- Extra test pages: do you have real scanned or non-banded balance sheets? Without them I can only simulate scans by rasterising this page.
- Review of the ground truth: have you looked at the ground truth JSON? Every metric so far is measured against it.

If you agree, I'll write the plan for the band-free core and the OCR routing, with approach, packages, risks and tests, and wait for your approval before coding.

> - large downloads are ok. proceed with them. but make sure to delete the useless or redundant ones later on, when they aren't needed anymore.
- Well i did share a link with you. And if we cannot find more, then how about we use it on other financial statements instead? I am sure all the finance statements have some kind of rule to verify them. We can use that.
- Where is the ground truth? I will have a look at that. And what kind of review do u want me to do?

● I'll answer the ground-truth question concretely and check what the existing parser does on the other statements in your link's PDF, since that determines the plan. Let me look at pages 10, 11, 14 and 15 (income statement, comprehensive income, cash flow).

Ran 1 shell command (ctrl+o to expand)

● I've noticed the other statements parse with plausible structure, but the income statement validator shows 2 FAILs. I'll dig into whether these are parser errors or rule issues before answering your question about reusing other statements. · summary

Ran 1 shell command (ctrl+o to expand)

● The P&L FAILs trace to my validator wrongly applying the balance-sheet sibling-sum rule to a flat income statement, while your idea checks out - all 20 statement-specific identities, including cross-statement ties, hold within ±1. Now I'll check the company IR page for more test material. · summary

● Fetch(https://www.yamato-in.co.jp/ir/)
  └ Received 30.9KB (200 OK)

● This IR page has several same-company filings worth using: quarterly 決算短信, three 訂正 correction notices, a 有価証券報告書 with a different layout, and a 半期報告書. I'll download the 9 relevant PDFs into separate directories (since they're untrusted) and verify they're valid PDFs. · summary

Ran 1 shell command (ctrl+o to expand)

● All 10 are real PDFs. Now a survey: which pages are balance sheets in each, and what does the current parser do with them? This tells us how much layout variation we actually have.

Made 1 scratchpad edit +39, ran 1 shell command (ctrl+o to expand)

● Those "CRASH" lines are my survey script, not the parser: python -I ignores PYTHONIOENCODING, so printing a Japanese statement name failed. Re-running with UTF-8 mode on.

Ran 1 shell command (ctrl+o to expand)

● I found a broader test set than expected—about 22 balance-sheet pages across 10 documents. The parser handles all the 決算短信-style pages fine, but the 有価証券報告書 pages show real failures, so I'll dig into those next. · summary

Read 1 file, ran 1 shell command (ctrl+o to expand)

● Found the root cause: footnote markers like ※1 attached to numbers (e.g., ※1 2,121,360) break my parser, which silently reads them as 0 — a real layout gap the accounting checks correctly flagged, and I'll add a loud warning for unreadable numeric cells. Now I'll generate a readable review sheet instead of raw JSON to go through the 43 ground-truth rows. · summary

Made 1 scratchpad edit +65, ran 1 shell command (ctrl+o to expand)

● Now the corpus provenance note and the plan document.

● Write(data¥corpus¥README.md)
  ⎿ Wrote 21 lines to data¥corpus¥README.md
     1 # Test corpus (downloaded, untrusted)
     3 Downloaded 2026-10-23 from the company's public IR page https://
       www.yamato-in.co.jp/ir/
     4 (ヤマト モビリティ & Mfg.株式会社, code 7886). Each file sits in
     5 untrusted data: nothing here is executed, and Python that reads
       them runs with `python -I`.
     7 | Directory | Document | Balance-sheet pages (0-based) |
     8 |---|---|---|
     9 | ir202603q4 | FY2026 full-year 決算短信 (consolidated, JGAAP) |
    10 | ir202603q3 | Q3 決算短信 | 5, 6 |
    … +21 lines (ctrl+o to expand)

● Write(docs¥plans¥phase2-plan.md)
  ⎿ Wrote 87 lines to docs¥plans¥phase2-plan.md
     1 # Phase 2 plan: layout-independent core, hybrid (OCR) routing, s
     2
     3 > Status: **PROPOSED, awaiting approval.** No code for this phas
       e has been written. (AGENT.md: plan first.)
       dated balance sheet, possibly with slight
     5 > layout variation and possibly scanned, extract the table prese
     6 > units and accounting signs. No hard-coded coordinates.
     7
     8 ## 1. Evidence that shapes the plan
    10 | Finding | Source | Consequence |
    … +139 lines (ctrl+o to expand)

● Two numbers in the plan need correcting against the survey output: 9 of 10 documents parse with 0 FAIL (not 8), and the survey found 23 balance-sheet pages (not 22).

Ran 1 shell command (ctrl+o to expand)
● I've written the Phase 2 plan and need your approval before I start. I haven't installed any OCR packages yet, because AGENT.md asks for plan approval first. I also found two problems in my own validation code along the way.

1. Ground truth

The file is tests/fixtures/ground_truth/balance_sheet_after_correction.json. Reading raw JSON is a poor way to review it, so I generated a readable sheet: docs/review/ground-truth-review.md.

What to review (about 10 minutes): open the PDF next to the sheet and compare against the PDF, not the Excel output, since a mistake the parser and I share would otherwise stay hidden. Check each of the 43 rows for:
- the label spelling
- the indent depth (shown as > marks)
- every digit, and the △ and ー marks
- the row type
- the two underlined cells (*)
- the headers and the 千円 unit

What to send back: "all correct", or something like "row 11: current should be △0". I'll fix it and re-run the metrics.

Until you've done this, every metric so far rests on a ground truth I wrote myself.

2. Using other statements

Your idea works. The parser already reads the income statement, comprehensive income and cash flow pages from complete_pdf.pdf. I checked 20 statement-specific identities on the parsed numbers, and all hold within ±1 rounding:
- the P&L chain (gross profit = sales − cost of sales, and so on down to net profit)
- comprehensive income = net profit + other comprehensive income
- cash-flow ending cash = balance sheet cash
- cash-flow pre-tax profit = income-statement pre-tax profit

It also exposed a bug in my validator. It applied the balance-sheet rule (a total equals the sum of its siblings) to the flat income statement. That gave 2 false FAILs, which were my bug and not parser errors. So check is only trustworthy on balance sheets today, and the plan fixes this first with per-statement rule sets.

3. More test material

The link was complete_pdf.pdf, which I'd already been using. I also went to the Yamato IR page you mentioned and downloaded 10 more PDFs, each into its own directory (provenance in data/corpus/README.md). They contain 23 balance-sheet pages:
- the quarterly and full-year 決算短信
- three correction notices, which should give more before/after pairs
- a half-year report and an annual securities report

Results from the current parser:

Documents	Result	
9 of 10	Parse, 0 failed checks	
Annual securities report	11 failed checks on one page	

The annual report prints footnote markers beside the numbers (※1 2,121,360). My parser glues the marker into the cell, can't read it as a number, and silently counts it as 0. The checks caught it. That is a real layout variation, and the parser should have warned loudly about an unreadable number.

It is all the same company, so layout variety is limited. No XBRL was offered on the IR page.

Recommended next step

Per your goal, the order I propose is:
- A. Statement-aware validation: about 1 day, no new packages. This is the oracle for everything after it.
- B. A layout-independent core: it works from words alone, with drawings as optional hints. About 2 days, no new packages. When I removed the graphics from the sample page, the parser refused, even though the text was intact. OCR output is words with boxes and no drawings, so the same core serves scans.
- C. Hybrid routing: triage picks the text layer or OCR, and the accounting checks decide when they disagree. I'd compare two Japanese OCR engines on a rasterised copy of the page.
- D. Evaluation on the corpus: pages that pass all checks serve as automatic "silver" labels, which score the OCR route at no labelling cost. These are not gold labels, so your review sheet is how we calibrate them.

Decisions I need from you

1. Approve the order A → B → C → D. A and B need no new packages, so they can start as soon as you say so.
2. Approve the OCR packages:
   - PaddleOCR: with paddlepaddle.
   - YomiToku: built for Japanese, and reportedly non-commercial, which is fine for personal use but I'll verify the licence.
   - opencv-python-headless: for rasterised-page preprocessing.
   - Tesseract: it needs a Windows installer with elevated rights, so I'd give you manual steps or skip it. Say which.
3. Confirm the cleanup rule: every install goes in a ledger (package, size, reason). After the comparison I'll propose what to remove and delete nothing without your OK. Torch-based installs are typically several GB, and I'll report real sizes.

The full plan, with risks and tests, is in docs/plans/phase2-plan.md.

> ok all are approved. u can procced to next steps. Alongside this I want you to generate a good, minimalistic UI where I can upload any PDF and it gives me the output.
This can be used for testing.
Procced ahead.

● All approved, so I'm starting. I'll work in the planned order (A → B → C → D) and build the upload UI alongside. First I'll re-read the validator as it stands on disk, since step A restructures it.

Read 2 files (ctrl+o to expand)

● I've got the full picture of the validator. Step A is a restructure, so I'll start with the model changes: a statement kind, and a WARN status for advisory ties.

Ran 1 shell command (ctrl+o to expand)

● Now statement-kind detection and the declarative rule file.

Ran 1 shell command (ctrl+o to expand)

● Write(src¥fin_table_parser¥rules¥statement_rules.yaml)
  ⎿ Wrote 42 lines to src¥fin_table_parser¥rules¥statement_rules.yaml
     1 # Accounting identities per statement type, in a small readable
     2 #
        and - are required)
     4 #   A TERM or TARGET may list alternative labels with "|" (first
     5 #   them and has a value is used). A trailing "?" marks an optio
     6 #   Labels are matched on their start, after NFKC normalisation
     7 #   "営業利益" matches "営業利益又は営業損失（△）".
     8 #
       ved: every figure is rounded
       y about one unit per figure.
    … +70 lines (ctrl+o to expand)

● Statement names currently keep stray brackets ((連結損益計算書),
  1【連結貸借対照表】). I'll normalise that too, then write the rule engine.

Ran 1 shell command (ctrl+o to expand)

● The ¥1 backreference got eaten by the shell quoting (it became a control
  character). I'll fix that line directly.

Ran 1 shell command (ctrl+o to expand)

● Titles normalise correctly now. Next, the rule engine that loads the YAML and
  evaluates formulas and cross-statement ties.

● Write(src¥fin_table_parser¥rules¥__init__.py)
  ⎿ Wrote 5 lines to src¥fin_table_parser¥rules¥__init__.py
     1 """Declarative accounting rules (see ``statement_rules.yaml``) an
       d the engine that evaluates them."""
     2
     3 from .engine import check_formulas, check_ties, load_rules
     4
     5 __all__ = ["check_formulas", "check_ties", "load_rules"]

● Write(src¥fin_table_parser¥rules¥engine.py)
  ⎿ Wrote 241 lines to src¥fin_table_parser¥rules¥engine.py
     1 """Evaluate the formulas and cross-statement ties of ``statement_
       rules.yaml``.
     3 A formula is looked up in *all* tables of the right statement kin
       d (a cash-flow statement spans two
     4 pages), and columns of different tables are paired by period end
     5 A rule that does not apply (its target row is absent) produces no
        finding, so a page is never
     6 blamed for a rule about a row it does not contain.
     8
     9 from __future__ import annotations
    … +258 lines (ctrl+o to expand)

● I spotted a bug: my sign-flip logic for negative loss values would incorrectly
  also match "特別損失合計" (total extraordinary losses), which should stay
  positive. I'll limit the flip to profit-line labels only and update the
  validator accordingly. · summary

Ran 2 shell commands (ctrl+o to expand)

● Two fixes: import order, and my YAML put ? inside an alternatives list, where
  only the last one is recognised. I'll make each optional cash-flow adjustment
  its own term.

Ran 1 shell command (ctrl+o to expand)

● All existing tests pass. Now I'll run the full validator across all statements
  in the filing (balance sheet, P&L, comprehensive income, cash flow) and
  review every non-pass result. · summary

Ran 1 shell command (ctrl+o to expand)

● All checks pass with 0 failures across 6 pages and the P&L false positives are
  resolved, but the second cash-flow page lacks a title so its labels weren't
  matched and its formulas never ran. I'll fix the detection logic to catch that
  page. · summary

Ran 1 shell command (ctrl+o to expand)

● The filing now passes all 80 applicable checks cleanly. Next, I'll add tests
  covering rule parsing, kind detection, the P&L false-FAIL regression, and the
  silent-zero hazard. · summary

● Write(tests¥unit¥test_rules.py)
  ⎿ Wrote 125 lines to tests¥unit¥test_rules.py
     1 """Rule parsing, kind detection and formula/tie evaluation on sm
     2
     3 import pytest
     5 from fin_table_parser.labels import classify_row, detect_kind
     6 from fin_table_parser.models import Row, Table

```
    n_number
 8 from fin_table_parser.rules import check_formulas, check_ties, l
 9 from fin_table_parser.rules.engine import parse_expr, signed_val
   ue
   arize, validate_tables
… +140 lines (ctrl+o to expand)
```

Ran 1 shell command (ctrl+o to expand)

● All 77 tests pass for step A. Now I'll look at how footnote markers actually appear in the annual report's text layer before designing the band-free core for step B. · summary

Ran 2 shell commands (ctrl+o to expand)

● Found that the footnote marker is fused into the number itself (e.g. ※12,121,360), using a full-width 1 before an ASCII digit—so it needs to be stripped before NFKC folding to avoid misreading as 12,121,360. Next, I'll update the number parser to handle note markers and move shared helpers into a common module. · summary

Ran 1 shell command (ctrl+o to expand)

● The marker now splits off correctly, and the ambiguous ※12,121,360 form gets flagged by the unparsed-cell check instead of silently guessed. Next I'll move on to the shared module and the band-free extractor. · summary

Ran 1 shell command (ctrl+o to expand)

● The refactor is clean and nothing regressed. Now the band-free extractor itself.

● Write(src¥fin_table_parser¥strategies¥textgeom.py)
```
└ Wrote 222 lines to src¥fin_table_parser¥strategies¥textgeom.py
   1 """Band-free strategy: rows, columns and hierarchy from the word
   2
   3 Input is a list of words with boxes, which is exactly what OCR p
   4 born-digital PDFs without row bands and scanned pages. Drawings
     are optional hints (underlines).
   6 Idea
   7 ----
     e.
   9 * Value columns: amounts are right-aligned, so the right edges o
  10   few x positions. Those clusters are the columns.
… +247 lines (ctrl+o to expand)
```
● I left some debris in that draft (a debug helper, a type-import hack, and an unexplained constant for single-column tables). Cleaning it up before testing.

Ran 1 shell command (ctrl+o to expand)

● Now the key experiment: the band-stripped page (graphics removed, 132 words) through the new text-only strategy, scored against the ground truth.

Ran 1 shell command (ctrl+o to expand)

● The text-only strategy nails 100% of structural metrics on the first run, with only expected misses—no underlines and a footer artifact (- 4 -) wrongly parsed as a row. I'll fix the latter by requiring amounts to align over a value column. · summary

Ran 1 shell command (ctrl+o to expand)

● All 43 rows now match the ground truth, aside from the two underline flags that can't survive without graphics. Next I'll build a reusable table-diff function to compare the band-free results against the band-based reference across every statement page. · summary

Ran 1 shell command (ctrl+o to expand)

● Good news: row labels, levels, types, and cell values match across strategies, but headers differ since my text strategy wrongly pulls in the running page header instead of just the block above the amounts. I'll fix that header logic, then check the row-count mismatch on CF p. 13. · summary

Ran 1 shell command (ctrl+o to expand)

● 27 of 28 pages are now identical, including the annual-report pages with ※ markers. One page (cash flow p. 13) still differs by 2 rows, and mypy reports 2 errors. Let me look at both.

Ran 1 shell command (ctrl+o to expand)

● I found two issues: a wrapped cash-flow label where the amounts are vertically centered across two lines, which I'll fix by grouping lines into row blocks based on their gaps, plus mypy errors in my new diff helper that I'll also address. · summary

Ran 1 shell command (ctrl+o to expand)

● Both strategies now match on all 28 comparable pages. I'll apply a quick lint

fix and lock this in with tests covering the band-stripped page, footnote markers, wrapped labels, and corpus agreement. · summary

Ran 1 shell command (ctrl+o to expand)

● Write(tests¥integration¥test_text_strategy.py)
  ⎿ Wrote 126 lines to tests¥integration¥test_text_strategy.py
     1 """The band-free (words only) strategy: the basis for OCR and fo
       r pages without shaded rows."""
     2
     4
     5 import pymupdf
     6 import pytest
     7
       truth, table_diff
     9 from fin_table_parser.labels import label_key
    10 from fin_table_parser.pipeline import parse_page
    … +136 lines (ctrl+o to expand)

Ran 1 shell command (ctrl+o to expand)

● Step B is complete: the band-free core matches the band-based parser on all 28 pages with 92 tests passing. Moving to step C now, I'll install each OCR engine into its own isolated virtual environment for easy cleanup later, and I'm checking disk space before starting those installs in the background. · summary

Ran 1 shell command (ctrl+o to expand)

● 58 GB free, venvs created. Starting both installs in the background (these are large and may take a while) while I build the UI.

Ran 2 shell commands (ctrl+o to expand)

● While the install runs, I'll start building the UI, beginning with testable logic for parsing the whole document—detecting which pages contain statement tables and why others don't—plus support for in-memory PDFs and page previews. · summary

Ran 1 shell command (ctrl+o to expand)

● Write(src¥fin_table_parser¥document.py)
  ⎿ Wrote 70 lines to src¥fin_table_parser¥document.py
     1 """Whole-document parsing: which pages hold a statement table, a
       nd why the others do not.
     3 This is the layer the UI and any batch job use. It never raises
       for a bad page; each page gets a
     4 ``PageResult`` that carries either a table or the reason there i
     5 """
     6
     7 from __future__ import annotations
     9 from collections.abc import Callable, Sequence
    10 from dataclasses import dataclass
    … +76 lines (ctrl+o to expand)

● I'll now run the document layer across all pages of both the 19-page earnings summary and the 109-page annual report to check which pages it flags as statement tables, making sure prose pages aren't misidentified as tables. · summary

Ran 1 shell command (ctrl+o to expand)

● The statement-page detection is fast but misclassifies equity-change pages as balance sheets and flags note tables that merely reference a statement name—causing the spurious FAILs. I'll fix the classification logic so only actual statements are matched. · summary

Ran 1 shell command (ctrl+o to expand)

● The totals didn't move, so my fix isn't the whole story. Let me look at which findings actually FAIL and which kinds the pages now get.

Ran 1 shell command (ctrl+o to expand)

● The remaining failures are all unparsed_cell cases from out-of-scope pages (grid-ruled equity statements and multi-amount summary tables), confirming the new check correctly flags unreliable output instead of silently passing it. Now I'll build a minimal upload-and-view UI with Streamlit, starting with installation. · summary

Ran 1 shell command (ctrl+o to expand)

● Streamlit is in. I'll keep the UI thin: all the logic (data frames, summaries, page-range parsing) goes in a plain testable module, and the Streamlit file is only layout.

● Write(src¥fin_table_parser¥ui_model.py)
  ⎿ Wrote 128 lines to src¥fin_table_parser¥ui_model.py
     1 """Data shaping for the UI (no Streamlit import, so it can be un
       it-tested)."""

```
     2
     3 from __future__ import annotations
     4
     6 from collections.abc import Sequence
     7
     8 import pandas as pd
     9
    10 from .document import PageResult
   … +135 lines (ctrl+o to expand)
```

● Write(src¥fin_table_parser¥app.py)
  ⎿ Wrote 179 lines to src¥fin_table_parser¥app.py

```
     1 """Streamlit UI: upload a PDF, get its statement tables, checks
       and an Excel file.
     2
       in_table_parser/app.py)
     4 """
     5
     6 from __future__ import annotations
     8 import streamlit as st
     9
    10 from fin_table_parser.document import PageResult, parse_document
   … +200 lines (ctrl+o to expand)
```

● Now the CLI entry point, a privacy-friendly Streamlit config, and tests. For an end-to-end UI test, I'll use Streamlit's own test harness with a stand-in for the uploaded file.

  Ran 1 shell command (ctrl+o to expand)

● The shell choked on that long heredoc again. I'll write each file with the file tool instead.

● Write(.streamlit¥config.toml)
  ⎿ Wrote 9 lines to .streamlit¥config.toml

```
     1 [browser]
     2 gatherUsageStats = false
     3
     4 [server]
     5 maxUploadSize = 200
     6
     7 [theme]
     8 base = "light"
     9 primaryColor = "#1F4E78"
```

● Update(src¥fin_table_parser¥cli.py)
  ⎿ Added 8 lines

```
    60      return 0 if scores.page_exact else 1
    61
    62
    63 +def _cmd_ui(args: argparse.Namespace) -> int:
    64 +    import subprocess
    65 +
    66 +    app = Path(__file__).with_name("app.py")
    67 +    cmd = [sys.executable, "-m", "streamlit", "run", str(app),
       +"--server.port", str(args.port)]
    68 +    return subprocess.call(cmd)
    69 +
    70 +
    71  def truth_page(truth_path: str) -> int:
    72      import json
    73
```

● Update(src¥fin_table_parser¥cli.py)
  ⎿ Added 3 lines

```
    92      e.add_argument("-o", "--output", required=True, help="Mark
        down report path")
    93      e.add_argument("--workdir", default="output")
    94      e.set_defaults(func=_cmd_evaluate)
    95 +    u = sub.add_parser("ui", help="open the upload-a-PDF web p
       +age")
    96 +    u.add_argument("--port", type=int, default=8501)
    97 +    u.set_defaults(func=_cmd_ui)
    98      args = ap.parse_args(argv)
    99      try:
   100          return int(args.func(args))
```

● Write(tests¥unit¥test_ui_model.py)
  ⎿ Wrote 75 lines to tests¥unit¥test_ui_model.py

```
     1 import pytest
     2
     3 from fin_table_parser.document import PageResult
     4 from fin_table_parser.pipeline import parse_page
     5 from fin_table_parser.ui_model import (
     6     INDENT,
     7     findings_frame,
     8     parse_page_spec,
     9     select_results,
    10     skipped_pages,
   … +70 lines (ctrl+o to expand)
```

● Write(tests¥integration¥test_document_and_app.py)
  ⎿ Wrote 112 lines to tests¥integration¥test_document_and_app.py
     1 """Whole-document parsing and the Streamlit page, driven end to
       end."""
     2
     4
     5 import pytest
     6 from streamlit.testing.v1 import AppTest
     7
     9 from fin_table_parser.document import parse_document, validate_r
       esults
    10 from fin_table_parser.export import workbook_bytes
    … +116 lines (ctrl+o to expand)

  Ran 1 shell command (ctrl+o to expand)

● Found a genuine bug: untitled continuation pages were getting mislabeled by
  sheet name ("Page 9") instead of their actual statement. I'll fix this by
  storing the statement explicitly in the sheet rather than inferring it from
  the title cell. · summary

  Ran 1 shell command (ctrl+o to expand)

● 118 tests pass. Now let me actually run the UI and confirm it serves.

  Ran 1 shell command (ctrl+o to expand)

● The server works, but Streamlit is bound to all network interfaces by default,
  exposing the financial-document upload page to the local network. I'll
  restrict it to localhost and restart. · summary

  Ran 2 shell commands (ctrl+o to expand)

● The server is stopped, but the config edit didn't apply. Doing it directly:

● Update(.streamlit¥config.toml)
  ⎿ Added 2 lines
     2 gatherUsageStats = false
     3
     4 [server]
     5 +# The page handles financial documents: keep it reachable from t
       +his machine only.
     6 +address = "localhost"
     7 maxUploadSize = 200
     8
     9 [theme]

  Ran 1 shell command (ctrl+o to expand)

● Network is busy with three large installs, so I'll use the wait to start the
  install ledger you asked for (what was installed, why, and how to remove it).

● Write(docs¥worklog¥installed-packages.md)
  ⎿ Wrote 37 lines to docs¥worklog¥installed-packages.md
     1 # Installed packages ledger
     2
       g redundant is removed once it is no
     4 longer needed, and nothing is deleted without telling them. OCR
       candidates and the browser used for UI
       ect root, so removing one is deleting one
     6 folder and cannot break the main environment.
     7
     9
    10 | Package | Why | Status |
    … +40 lines (ctrl+o to expand)

  Ran 1 shell command (ctrl+o to expand)

● While the downloads finish, I'll build the OCR adapter layer—each engine runs
  in its own virtual environment, communicating with the main code via a worker
  script that returns JSON, keeping the core lightweight and engines easily
  swappable. · summary

● Write(src¥fin_table_parser¥ocr.py)
  ⎿ Wrote 109 lines to src¥fin_table_parser¥ocr.py
     1 """OCR route: turn a page image into words with boxes, the same i
       nput the text strategy uses.
     3 Engines (PaddleOCR, YomiToku, ...) live in their own virtual envi
       ronments so the main environment
     4 stays light and an engine can be removed by deleting one folder.
     5 through a worker script that prints JSON; ``OcrEngine`` is the on
       ly interface the rest of the
     6 package sees, so a test can use a stand-in.
     8
     9 from __future__ import annotations
    … +118 lines (ctrl+o to expand)

  Ran 1 shell command (ctrl+o to expand)

● OCR integration is working with no regressions, so while the engines finish

downloading I'll test the text strategy's robustness by simulating OCR imperfections—jittered boxes and split digits—and sweeping noise levels to find the breaking point. · summary

● Write(tests¥ocr_sim.py)
  ⎿ Wrote 46 lines to tests¥ocr_sim.py
     1 """Stand-in OCR engine for tests: the page's own words, degraded
       the way real OCR degrades them.
     3 Real OCR boxes are less exact than a text layer (jitter), and en
       gines often split a sign from
     4 its digits (``△`` + ``2,979``). This lets the routing and the to
     5 tested without a real engine.
     6 """
     7
     9
    10 import random
    … +45 lines (ctrl+o to expand)

  Ran 1 shell command (ctrl+o to expand)

● I found three issues: the text strategy tolerates up to ±2pt box jitter, a look-alike mapping bug incorrectly converts katakana long-vowel marks to dashes (corrupting "リース資産" labels), and the strategy fails to rejoin split minus signs from their digits. I'll fix the mapping bug and the sign-splitting issue now. · summary

  Ran 1 shell command (ctrl+o to expand)

● Both issues are fixed — exact matching now tolerates ±2 pt jitter and split signs get rejoined. Next I'll test the OCR route: when it's triggered, how it handles look-alikes, and its noise tolerance. · summary

● Write(tests¥integration¥test_ocr_route.py) ⎿ Wrote 99 lines to tests¥integration¥test_ocr_route.py
     1 """Hybrid routing with a stand-in OCR engine (no real engine nee
       ded)."""
     3 import pymupdf
     4 import pytest
     5
       truth
     7 from fin_table_parser.ocr import clean_text, ocr_geometry
     8 from fin_table_parser.pdf_io import load_page
       ce
    10 from fin_table_parser.triage import UnsupportedPage
    … +102 lines (ctrl+o to expand)

  Ran 1 shell command (ctrl+o to expand)

● 136 tests pass. Let me check how the three background installs are doing.

  Ran 1 shell command (ctrl+o to expand)

● While the large wheel downloads finish, I'll build the bake-off harness: using accounting-verified text-layer parses as silver references, I'll render pages as both clean/low-res and skewed JPEG images to simulate scans, then run and score each OCR engine with the same metrics as before. · summary

  Ran 1 shell command (ctrl+o to expand)

● Now the bake-off module itself.

● Write(src¥fin_table_parser¥ocr_eval.py)
  ⎿ Wrote 217 lines to src¥fin_table_parser¥ocr_eval.py
     1 """OCR bake-off: how well does each engine reproduce a trusted p
     2
     3 Reference ("silver"): balance-sheet pages whose text-layer parse
     4 Each page is rendered as an image under three conditions and the
       engine's words are run through the
       re differences in recognition.
     6 Scores are the usual cell / structure metrics plus a character e
     7 accounting checks notice the engine's mistakes.
     8 """
    10 from __future__ import annotations
    … +254 lines (ctrl+o to expand)

  Ran 1 shell command (ctrl+o to expand)

● 22 silver pages: a good reference set. There's also a lint of my own making: ocr_eval imports PyMuPDF directly, breaking the "only pdf_io touches the PDF library" rule. Fixing that properly.

  Ran 3 shell commands (ctrl+o to expand)

● Installs are active (pip's cache has grown to 563 MB), just slow on this connection. I'll use the wait to write the concept docs for what's finished.

● Write(docs¥explanations¥03-band-free-core-and-ocr-route.md)
  ⎿ Wrote 81 lines to docs¥explanations¥03-band-free-core-and-ocr-route.md
     1 # The band-free core and the OCR route
     2
     3 Audience: a developer new to the project. Code: [strategies/textg

```
     4 [ocr.py](../../src/fin_table_parser/ocr.py), [pipeline.py](../../
       src/fin_table_parser/pipeline.py).
     5
     6 ## Why it exists
     8 The first parser (`strategies/banded.py`) reads rows and columns
       from the shaded row bands a page draws. Two
     9 facts made that insufficient:
    … +111 lines (ctrl+o to expand)

● Write(docs¥explanations¥04-the-ui.md)
  └ Wrote 40 lines to docs¥explanations¥04-the-ui.md
     1 # The upload UI
     2
        [ui_model.py](../../src/fin_table_parser/ui_model.py)
     4 (data shaping, unit-tested), [document.py](../../src/fin_table_p
       arser/document.py) (whole-document parsing).
     6 ## Start it
     7
     8 ```
       t:8501
    10 python -m fin_table_parser ui --port 8765
    … +47 lines (ctrl+o to expand)

● I'll wire up the OCR route to the document layer and UI so scanned pages work
  from the upload page once an engine is ready, including a "force OCR" toggle
  to run OCR on any PDF. · summary

  Ran 2 shell commands (ctrl+o to expand)

● Still downloading (PaddlePaddle is in; the rest of its stack and torch are
  coming). While waiting I'll add a docs index so the growing documentation is
  navigable.

● Write(docs¥README.md)
  └ Wrote 41 lines to docs¥README.md
     1 # Documentation index
     2
     3 Goal: given a PDF page with a Japanese consolidated balance sheet
     4 table with row labels, values, columns, hierarchy, units and acco
       unting signs, with no hard-coded coordinates.
     5
     6 Start with the project status in [worklog/phase2-worklog.md](work
     7 documents in the order below.
     8
     9 ## Explanations (how it works, for a new developer)
    … +53 lines (ctrl+o to expand)

● I've built a lot since the last review, and the OCR installs are still
  pending, so this is a good moment to get an outside read on the approach
  before the bake-off.

● Advising using Opus 5.5
  └ √ Advisor has reviewed the conversation and will apply the feedback

● I'll incorporate the review feedback—robust worker protocol, batching, shared
  extraction logic, fairer sampling, de-skewing, OCR triggers for embedded
  tables, localhost flags, and broader cleanup tracking—before building the
  bake-off. Starting now with the OCR layer's batch interface and file-based
  protocol. · summary

● Write(src¥fin_table_parser¥ocr.py)
  └ Error writing file

● Background command "Install a headless browser in its own venv to check the UI
visually" completed (exit code 0)

● The browser install finished. The write failed because ruff format had touched
  the file since I last read it. Let me re-read it and then apply the rewrite.

  Read 1 file (ctrl+o to expand)

● Write(src¥fin_table_parser¥ocr.py)
  └ Added 60 lines, removed 22 lines
     1 -"""OCR route: turn a page image into words with boxes, the same
       - input the text strategy uses.
     1 +"""OCR route: turn page images into words with boxes, the same
       +input the text strategy uses.
     2
     3  Engines (PaddleOCR, YomiToku, ...) live in their own virtual en
       vironments so the main environment
     4  stays light and an engine can be removed by deleting one folder
       . The main code talks to an engine
     5 -through a worker script that prints JSON; ``OcrEngine`` is the
       -only interface the rest of the
     6 -package sees, so a test can use a stand-in.
     5 +through a worker script; ``OcrEngine`` is the only interface th
       +e rest of the package sees, so a
     6 +test can use a stand-in.
     7 +
     8 +Worker protocol (see ``tools/ocr``): ``python worker.py --out r
```

```
   +esult.json image1 image2 ...``.
 9 +The worker writes ONE JSON file and sends all its logging to st
   +derr, so library banners can never
10 +corrupt the result. Several images go in one call so that model
   + loading is paid once and can be
11 +reported separately from per-page recognition time.
12  """
13
14  from __future__ import annotations
...
18  import subprocess
19  import sys
20  import tempfile
16 -from dataclasses import dataclass
21 +from collections.abc import Sequence
22 +from dataclasses import dataclass, field
23  from pathlib import Path
24  from typing import Protocol
25
...
28
29  OCR_DPI = 300
30  PROJECT_ROOT = Path(__file__).resolve().parents[2]
25 -WORKER_DIR = PROJECT_ROOT / "tools" / "ocr"
31
32  # OCR engines often return a visually similar character instead
    of the accounting sign.
33  # A bare dash-like token means "nil"; the same characters insid
   e a word are NOT touched (the
...
 47      y1: float
 48
 49
 50 +@dataclass
 51 +class OcrBatch:
 52 +    """Words for each image, plus timing: model load is report
   +ed apart from recognition."""
 53 +
 54 +    pages: list[list[PixelWord]]
 55 +    load_seconds: float = 0.0
 56 +    page_seconds: list[float] = field(default_factory=list)
 57 +
 58 +
 59  class OcrEngine(Protocol):
 60      name: str
 61
 48 -    def read_image(self, png: bytes) -> list[PixelWord]: ...
 62 +    def read_images(self, images: Sequence[bytes]) -> OcrBatch
   +: ...
 63
 64
 65  class SubprocessEngine:
 52 -    """Runs ``<venv python> <worker script> <image.png>`` and
   -parses the JSON it prints."""
 66 +    """Runs ``<venv python> <worker> --out <json> <images...>`
   +` and reads the JSON file."""
 67
 54 -    def __init__(self, name: str, python: Path, script: Path,
   -timeout: float = 600) -> None:
 68 +    def __init__(self, name: str, python: Path, script: Path,
   +timeout: float = 3600) -> None:
 69          self.name = name
 70          self.python = python
 71          self.script = script
 72          self.timeout = timeout
 73
 60 -    def read_image(self, png: bytes) -> list[PixelWord]:
 74 +    def read_images(self, images: Sequence[bytes]) -> OcrBatch
   +:
 75          with tempfile.TemporaryDirectory() as tmp:
 62 -            image = Path(tmp) / "page.png"
 63 -            image.write_bytes(png)
 76 +            paths = []
 77 +            for i, data in enumerate(images):
 78 +                suffix = ".jpg" if data[:2] == b"¥xff¥xd8" els
   +e ".png"
 79 +                path = Path(tmp) / f"page{i:03d}{suffix}"
 80 +                path.write_bytes(data)
 81 +                paths.append(str(path))
 82 +            out = Path(tmp) / "result.json"
 83              done = subprocess.run(
 65 -                [str(self.python), "-X", "utf8", str(self.scri
   -pt), str(image)],
 84 +                [str(self.python), "-X", "utf8", str(self.scri
   +pt), "--out", str(out), *paths],
 85                  capture_output=True,
 86                  text=True,
 87                  encoding="utf-8",
 88                  timeout=self.timeout,
 89                  check=False,
```

```
 90              )
 72 -        if done.returncode != 0:
 73 -            raise RuntimeError(f"{self.name} worker failed: {d
    -one.stderr.strip()[-400:]}")
 74 -        payload = done.stdout[done.stdout.index("[") :]  # eng
    -ines may print banners first
 75 -        return [PixelWord(**w) for w in json.loads(payload)]
 91 +            if done.returncode != 0 or not out.exists():
 92 +                raise RuntimeError(f"{self.name} worker failed
    +: {done.stderr.strip()[-600:]}")
 93 +            payload = json.loads(out.read_text(encoding="utf-8
    +"))
 94 +        pages = [[PixelWord(**w) for w in page["words"]] for p
    +age in payload["pages"]]
 95 +        return OcrBatch(
 96 +            pages=pages,
 97 +            load_seconds=float(payload.get("load_seconds", 0.0
    +)),
 98 +            page_seconds=[float(p.get("seconds", 0.0)) for p i
    +n payload["pages"]],
 99 +        )
100
101
102  def available_engines(root: Path = PROJECT_ROOT) -> dict[str,
     SubprocessEngine]:
...
114      return found
115
116
117 +def read_one(engine: OcrEngine, png: bytes) -> list[PixelWord]
    +:
118 +    return engine.read_images([png]).pages[0]
119 +
120 +
121  def clean_text(text: str) -> str:
122      text = text.strip()
123      if text in _NIL_LOOKALIKES:
...
125      return _SIGN_LOOKALIKE.sub("△", text)
126
127
100 -def words_from_pixels(pixel_words: list[PixelWord], dpi: int)
    -> list[Word]:
128 +def words_from_pixels(pixel_words: Sequence[PixelWord], dpi: i
    +nt) -> list[Word]:
129      """Pixel boxes -> PDF points, dropping empty boxes."""
130      k = 72.0 / dpi
131      words = []
...
136      return words
137
138
139 +def geometry_from_words(
140 +    source: PdfSource, page_no: int, pixel_words: Sequence[Pix
    +elWord], dpi: int = OCR_DPI
141 +) -> PageGeometry:
142 +    width, height = page_size(source, page_no)
143 +    return PageGeometry(
144 +        page_no=page_no,
145 +        width=width,
146 +        height=height,
147 +        words=words_from_pixels(pixel_words, dpi),
148 +        fills=[],
149 +        rules=[],
150 +    )
151 +
152 +
153  def ocr_geometry(
154      source: PdfSource, page_no: int, engine: OcrEngine, dpi: i
     nt = OCR_DPI
155  ) -> PageGeometry:
156      """Rasterise a page, run OCR, and return it as a words-onl
     y ``PageGeometry``."""
157      png = render_page(source, page_no, dpi=dpi)
116 -    width, height = page_size(source, page_no)
117 -    words = words_from_pixels(engine.read_image(png), dpi)
118 -    return PageGeometry(
119 -        page_no=page_no, width=width, height=height, words=wor
    -ds, fills=[], rules=[]
120 -    )
158 +    return geometry_from_words(source, page_no, read_one(engin
    +e, png), dpi)
```

● Now the de-skew module, the shared OCR extraction function, and the image-coverage routing trigger.

● Write(src¥fin_table_parser¥strategies¥deskew.py)
  ⎿ Wrote 72 lines to src¥fin_table_parser¥strategies¥deskew.py
     1 """Estimate and undo page skew from word positions (used on the
     2

```
      le rotated. A rotation of 1 degree
    4 shifts the far end of a 450 pt wide row by about 8 pt, more than
    5 and amounts would land on different lines.
    7 Method: amounts are right-aligned, so in a straight page the rig
    8 column all share an x. In a rotated page they drift linearly wit
      h y. Try a range of drift slopes
       (largest sum of squared
      ws about a specific
  … +79 lines (ctrl+o to expand)

  Ran 1 shell command (ctrl+o to expand)

● Now the bake-off module needs the batch interface, the shared production
  function, load time reported separately, and one page per document. I'll
  rewrite it.

  Read 1 file (ctrl+o to expand)

● Write(src¥fin_table_parser¥ocr_eval.py)  └ Added 81 lines, removed 63 lines
      1  """OCR bake-off: how well does each engine reproduce a trusted
         parse from a picture of the page?
      2
      3 -Reference ("silver"): balance-sheet pages whose text-layer pars
        -e passes every accounting check.
      4 -Each page is rendered as an image under three conditions and th
        -e engine's words are run through the
      5 -same band-free table extractor, so differences between engines
        -are differences in recognition.
      6 -Scores are the usual cell / structure metrics plus a character
        -error rate and whether the
      7 -accounting checks notice the engine's mistakes.
      3 +Reference ("silver"): balance-sheet pages whose text-layer pars
        +e passes every accounting check, one
      4 +page per company document so that near-identical pages do not d
        +ominate. Each page is rendered as an
      5 +image under three conditions and the engine's words go through
        +``pipeline.parse_ocr_words``, the
      6 +same function the production OCR route uses. Scores are the usu
        +al cell / structure metrics, a
      7 +character error rate, and whether the accounting checks notice
        +the engine's mistakes.
      8 +
      9 +Timing: the model is loaded once per engine and condition (one
        +batch call). Load time is reported
     10 +apart from per-page recognition time.
     11  """
     12
     13  from __future__ import annotations
     14
     15  import statistics
     13 -import time
     16  from collections.abc import Sequence
     17  from dataclasses import dataclass, field
     18  from pathlib import Path
     19
     20  from .evaluate import GroundTruth, score_against_truth, truth_f
         rom_table
     21  from .labels import label_key
     20 -from .models import PageGeometry, Table
     21 -from .ocr import OcrEngine, words_from_pixels
     22 -from .pdf_io import page_size, page_texts, render_page
     23 -from .pipeline import parse_geometry, parse_page
     22 +from .models import Table
     23 +from .ocr import OcrEngine, geometry_from_words
     24 +from .pdf_io import page_texts, render_page
     25 +from .pipeline import parse_ocr_words, parse_page
     26  from .triage import UnsupportedPage
     27  from .validate import summarize, validate_tables
     28
     27 -CONDITIONS: dict[str, dict[str, float | int | None]] = {
     29 +Condition = dict[str, float | int | None]
     30 +CONDITIONS: dict[str, Condition] = {
     31      "clean (300 dpi)": {"dpi": 300, "rotate_deg": 0.0, "jpeg_qu
         ality": None},
     32      "scan-like (150 dpi, JPEG q40)": {"dpi": 150, "rotate_deg":
          0.0, "jpeg_quality": 40},
     33      "skewed (1 degree)": {"dpi": 300, "rotate_deg": 1.0, "jpeg_
         quality": None},
    ...
     46          return f"{self.pdf.parent.name}:p{self.page_no + 1}"
     47
     48
     46 -def find_silver_pages(pdfs: Sequence[Path], limit: int | None =
        - None) -> list[SilverPage]:
     47 -    """Balance-sheet pages that parse from the text layer with
        -no failing check."""
     49 +def find_silver_pages(
     50 +    pdfs: Sequence[Path], limit: int | None = None, one_per_doc
        +ument: bool = True
     51 +) -> list[SilverPage]:
```

```
 52 +    """Assets-half balance-sheet pages that parse from the text
    + layer with no failing check."""
 53      pages: list[SilverPage] = []
 54      for pdf in pdfs:
 50 -        candidates = [
 51 -            i
 52 -            for i, text in enumerate(page_texts(pdf))
 53 -            if "資産合計" in label_key(text) or "負債純資産合計
    -" in label_key(text)
 54 -        ]
 55 -        for i in candidates:
 55 +        found_here = 0
 56 +        for i, text in enumerate(page_texts(pdf)):
 57 +            if "資産合計" not in label_key(text):
 58 +                continue
 59              try:
 60                  table = parse_page(pdf, i)
 61              except UnsupportedPage:
...
 63              counts = summarize(validate_tables([table]))
 64              if table.kind == "balance_sheet" and counts["FAIL"]
      == 0 and len(table.rows) >= 20:
 65                  pages.append(SilverPage(pdf, i, table, truth_fr
    om_table(table)))
 66 +                found_here += 1
 67 +                if one_per_document:
 68 +                    break
 69      return pages[:limit] if limit else pages
 70
 71
...
 88      ok: bool
 89      error: str = ""
 90      cell_f1: float = 0.0
 85 -    cell_exact_share: float = 0.0
 91      label_acc: float = 0.0
 92      level_acc: float = 0.0
 93      type_acc: float = 0.0
...
  96      cer: float = 1.0
  97      failing_checks: int = 0
  98      wrong_cells: int = 0
  99 +    skew_degrees: float = 0.0
 100
 101
 102  def _cer(table: Table, truth: GroundTruth) -> float:
  97 -    """Character error rate over all amount cells, rows paired
     - by position when counts agree."""
 103 +    """Character error rate over all amount cells; 1.0 when th
    +e rows cannot be paired."""
 104      pred = [v.raw if v else "" for r in table.rows for v in r.
      values]
 105      ref = [v or "" for r in truth.rows for v in r.values]
 106      if len(pred) != len(ref):
...
 109      return errors / max(1, sum(len(r) for r in ref))
 110
 111
 106 -def run_one(
 107 -    engine: OcrEngine, page: SilverPage, name: str, cond: dict
     -[str, float | int | None]
 112 +def score_words(
 113 +    engine_name: str,
 114 +    page: SilverPage,
 115 +    condition: str,
 116 +    geom_words: Sequence[object],
 117 +    seconds: float,
 118 +    dpi: int,
 119  ) -> RunResult:
 109 -    dpi = int(cond["dpi"] or 300)
 110 -    t0 = time.time()
 120 +    geom = geometry_from_words(page.pdf, page.page_no, geom_wo
     +rds, dpi)  # type: ignore[arg-type]
 121      try:
 112 -        png = render_page(
 113 -            page.pdf,
 114 -            page.page_no,
 115 -            dpi=dpi,
 116 -            rotate_deg=float(cond["rotate_deg"] or 0.0),
 117 -            jpeg_quality=int(cond["jpeg_quality"]) if cond["jp
     -eg_quality"] else None,
 118 -        )
 119 -        width, height = page_size(page.pdf, page.page_no)
 120 -        geom = PageGeometry(
 121 -            page.page_no, width, height, words_from_pixels(eng
     -ine.read_image(png), dpi), [], []
 122 -        )
 123 -        table = parse_geometry(geom, page.pdf.name, "text")
 122 +        table = parse_ocr_words(geom, page.pdf.name)
 123      except Exception as exc:  # noqa: BLE001 - a failed run is
```

```
      a result, not a crash
125 -        return RunResult(
126 -            engine.name, page.name, name, time.time() - t0, Fa
    -lse, error=str(exc)[:200]
127 -        )
128 -    seconds = time.time() - t0
124 +        return RunResult(engine_name, page.name, condition, se
    +conds, False, error=str(exc)[:200])
125      scores = score_against_truth(table, page.truth)
126      counts = summarize(validate_tables([table]))
131 -    total_cells = sum(1 for r in page.truth.rows for v in r.va
    -lues if v is not None) or 1
132 -    m = scores.metrics
127 +    metrics = scores.metrics
128      return RunResult(
134 -        engine.name,
129 +        engine_name,
130          page.name,
136 -        name,
131 +        condition,
132          seconds,
133          True,
134          cell_f1=scores.cells.f1,
140 -        cell_exact_share=scores.cells.tp / total_cells,
141 -        label_acc=m["row.label"].value,
142 -        level_acc=m["row.level"].value,
143 -        type_acc=m["row.type"].value,
135 +        label_acc=metrics["row.label"].value,
136 +        level_acc=metrics["row.level"].value,
137 +        type_acc=metrics["row.type"].value,
138          rows_pred=scores.rows_pred,
139          rows_truth=scores.rows_truth,
140          cer=_cer(table, page.truth),
141          failing_checks=counts["FAIL"],
142          wrong_cells=scores.cells.fp + scores.cells.fn,
143 +        skew_degrees=float(table.meta.get("skew_degrees", 0.0)
    + or 0.0),  # type: ignore[arg-type]
144      )
145
146
147  @dataclass
148  class Bakeoff:
149      results: list[RunResult] = field(default_factory=list)
150 +    load_seconds: dict[tuple[str, str], float] = field(default
    +_factory=dict)
151
152      def summary(self, engine: str, condition: str) -> dict[str
     , float]:
153          rows = [r for r in self.results if r.engine == engine
     and r.condition == condition]
...
162              "level_acc": statistics.mean(r.level_acc for r in
     good) if good else 0.0,
163              "cer": statistics.mean(r.cer for r in good) if goo
     d else 1.0,
164              "seconds": statistics.mean(r.seconds for r in rows
     ) if rows else 0.0,
165 +            "load": self.load_seconds.get((engine, condition),
    + 0.0),
166              "wrong_pages": len(wrong),
167              "wrong_pages_flagged": sum(r.failing_checks > 0 fo
     r r in wrong),
168          }
169
170
171  def run_bakeoff(engines: Sequence[OcrEngine], pages: Sequence[
     SilverPage]) -> Bakeoff:
172 +    """One batch call per engine and condition: the model load
    +s once, pages are timed separately."""
173      out = Bakeoff()
174      for engine in engines:
175          for name, cond in CONDITIONS.items():
178 -            for page in pages:
179 -                out.results.append(run_one(engine, page, name,
    - cond))
176 +            dpi = int(cond["dpi"] or 300)
177 +            images = [
178 +                render_page(
179 +                    p.pdf,
180 +                    p.page_no,
181 +                    dpi=dpi,
182 +                    rotate_deg=float(cond["rotate_deg"] or 0.0
    +),
183 +                    jpeg_quality=int(cond["jpeg_quality"]) if
    +cond["jpeg_quality"] else None,
184 +                )
185 +                for p in pages
186 +            ]
187 +            try:
188 +                batch = engine.read_images(images)
```

```
189 +            except Exception as exc:  # noqa: BLE001 - the who
    +le call failed: every page fails
190 +                for p in pages:
191 +                    out.results.append(
192 +                        RunResult(engine.name, p.name, name, 0
    +.0, False, error=str(exc)[:200])
193 +                    )
194 +                continue
195 +            out.load_seconds[(engine.name, name)] = batch.load
    +_seconds
196 +            for p, words, seconds in zip(pages, batch.pages, b
    +atch.page_seconds, strict=True):
197 +                out.results.append(score_words(engine.name, p,
    + name, words, seconds, dpi))
198      return out
199
200
...
203      lines = [
204          "# OCR engine comparison",
205          "",
188 -        f"Reference ('silver'): {len(pages)} balance-sheet pag
    -es whose text-layer parse passes every accounting "
189 -        "check (" + ", ".join(p.name for p in pages) + ").",
190 -        "Each page is rendered as an image, read by the engine
    -, and the words go through the band-free table "
191 -        "extractor. Silver is not gold: a mistake the text lay
    -er and I share would be invisible here.",
206 +        f"Reference ('silver'): {len(pages)} balance-sheet pag
    +es, one per company document, whose text-layer parse "
207 +        "passes every accounting check (" + ", ".join(p.name f
    +or p in pages) + ").",
208 +        "Each page is rendered as an image, read by the engine
    +, and the words go through the production OCR "
209 +        "function (`pipeline.parse_ocr_words`: de-skew + words
    +-only extractor).",
210          "",
211 +        "Caveats: silver is not gold (a mistake the text layer
    + and the extractor share is invisible), and every "
212 +        "page comes from the same company, so layout variety i
    +s small.",
213 +        "",
214      ]
215      for cond in CONDITIONS:
216          lines += [
217              f"## {cond}",
218              "",
198 -            "| Engine | Cell F1 | Pages exact | Label acc. | I
    -ndent acc. | Char error rate | s/page | Wrong pages caught by
    -checks | Failed runs |",
199 -            "|---|---|---|---|---|---|---|---|---|---|",
219 +            "| Engine | Cell F1 | Pages exact | Label acc. | I
    +ndent acc. | Char error rate | s/page | model load (s) | Wrong
    + pages caught by checks | Failed runs |",
220 +            "|---|---|---|---|---|---|---|---|---|---|---|",
221          ]
222          for e in engines:
223              s = bake.summary(e, cond)
224              lines.append(
225                  f"| {e} | {s['cell_f1']:.1%} | {int(s['pages_e
     xact'])}/{int(s['pages'])} | {s['label_acc']:.1%} "
205 -                f"| {s['level_acc']:.1%} | {s['cer']:.2%} | {s
    -['seconds']:.1f} | "
226 +                f"| {s['level_acc']:.1%} | {s['cer']:.2%} | {s
    +['seconds']:.1f} | {s['load']:.1f} | "
227                  f"{int(s['wrong_pages_flagged'])}/{int(s['wron
     g_pages'])} | {int(s['failed_runs'])} |"
228              )
229          lines.append("")
230      errors = [r for r in bake.results if not r.ok]
231      if errors:
211 -        lines += (
212 -            ["## Failed runs", ""]
213 -            + [f"- {r.engine} {r.page} {r.condition}: {r.error
    -}" for r in errors[:10]]
214 -            + [""]
215 -        )
232 +        lines += ["## Failed runs", ""]
233 +        lines += [f"- {r.engine} {r.page} {r.condition}: {r.er
    +ror}" for r in errors[:10]] + [""]
234      if sizes:
235          lines += ["## Disk footprint", ""] + [f"- {k}: {v}" fo
     r k, v in sizes.items()] + [""]
236      return "¥n".join(lines)
```

Ran 1 shell command (ctrl+o to expand)

● Tests pass, but the OCR-route tests should now exercise the production function and the new behaviours (skew, split signs, and the picture-of-a-table routing). Let me add those.

Ran 1 shell command (ctrl+o to expand)

● Four failures. I'll look at each rather than loosen tolerances. First the details of the two integration failures:

Ran 1 shell command (ctrl+o to expand)

● I found that at extreme box noise, the de-skew locks onto a spurious 0.2° rotation and shifts a row. I'll fix this properly by ignoring rotations under 0.3° (too small to matter) and selecting the center of the best plateau instead of its first edge, which also resolves the 0.15° bias seen in the unit test. · summary

Ran 1 shell command (ctrl+o to expand)

● The remaining failure is just a flaw in my test setup — the text-layer gate correctly flagged my English caption as lacking digits/Japanese and routed to OCR. I'll switch to a Japanese caption so the page has a genuine text layer without a table. · summary

Ran 2 shell commands (ctrl+o to expand)

● Since Japanese text has no spaces, PyMuPDF treats the caption as just 2 giant words, triggering the small-text-layer gate. I'll insert spaces between phrases so the page registers a proper text layer. · summary

Ran 3 shell commands (ctrl+o to expand)

● 147 tests pass. Now the visual UI check the reviewer asked for. I'll start the server on localhost and drive it with the headless browser: upload the real files and screenshot what comes out.

Made 1 scratchpad edit +47, ran 2 shell commands (ctrl+o to expand)

● Just my print statement hitting the Windows console encoding. Re-running in UTF-8 mode.

Ran 1 shell command (ctrl+o to expand)

● The end-to-end test works in a real browser—for the 109-page annual report, the default view found 4 balance-sheet pages with 68 checks passed and 0 failed. Now I'll look at the screens themselves. · summary

Read 2 files (ctrl+o to expand)

● I noticed a few issues: empty cells display "None", the Streamlit Deploy button shouldn't be there for a local tool, and tab names mix Japanese and English. I'll check the other views before making fixes. · summary

Read 2 files (ctrl+o to expand)

● I see several issues: "None" showing in empty cells, unformatted floats and truncated columns in the Checks tab, a misplaced "Deploy" button, and mixed Japanese/English tab names. I'll fix all of these now. · summary

Ran 1 shell command (ctrl+o to expand)

● 149 tests pass. Let me check where the OCR installs stand.

Ran 1 shell command (ctrl+o to expand)

● YomiToku is installing; PaddleOCR is still fetching packages. Meanwhile I'll restart the UI so it picks up the new config, and re-take the screenshots to confirm the fixes.

Ran 2 shell commands (ctrl+o to expand)

● Background command "Run the UI server in the background on localhost" failed with exit code 255

● That notification is just the old server I stopped deliberately. Re-taking the screenshots against the restarted server:

Read 2 files, ran 1 shell command (ctrl+o to expand)

● The Checks tab looks good now, but I found a bigger issue: a garbage summary table on page 2 is getting falsely ticked as a verified "Income statement." I'll fix this by only marking tables [ok] when an actual accounting check passes, and tightening the label-based fallback to require two distinct row labels matching statement vocabulary. · summary

Ran 1 shell command (ctrl+o to expand)

● 151 pass. I'll add the regression that matters most here: across a whole filing, the only pages that get a green [ok] must be the genuine statement pages.

Ran 1 shell command (ctrl+o to expand)

● Both documents now correctly flag only genuine statement pages (6 and 13 respectively), but my new unit test is failing and two real statements in the annual report (comprehensive income p. 46, standalone P&L p. 88) are missing checkmarks. I'll investigate both issues next. · summary

Ran 1 shell command (ctrl+o to expand)

● I identified two real issues from the corpus: the marker handling doesn't support multiple footnote markers, and the cost-of-goods-sold calculation incorrectly adds the closing stock instead of subtracting it. Now I'll check why my unit test is still passing despite this. · summary

Ran 1 shell command (ctrl+o to expand)

● That explains it: my str.replace edit to table_health silently didn't apply (the file had been reformatted since I last read it), so the old logic is still running. Checking:

Ran 1 shell command (ctrl+o to expand)

● Confirmed that edit didn't apply, and the app.py, labels.py, and test_rules.py edits may have failed the same way. I'll switch to the Edit tool, which fails loudly, and verify each earlier change, starting with table_health. · summary
● Update(src¥fin_table_parser¥ui_model.py)
  └ Added 17 lines, removed 6 lines
```
123
124
125  def table_health(table: Table) -> tuple[str, dict[str, int]]:
126 -    """('ok' | 'check' | 'unsupported', counts) for one table,
    - from its own checks."""
127 -    counts = summarize(validate_tables([table]))
128 -    unparsed = any(
129 -        f.check == "unparsed_cell" and f.status == "FAIL" for
    -f in validate_tables([table])
130 -    )
126 +    """('ok' | 'check' | 'unsupported' | 'unverified', counts)
    + for one table, from its own checks.
127 +
128 +    'ok' needs positive evidence: at least one accounting chec
    +k that passed (the numbers hang
129 +    together). A table on which no check could run is 'unverif
    +ied', never 'ok'.
130 +    """
131 +    findings = validate_tables([table])
132 +    counts = summarize(findings)
133 +    unparsed = any(f.check == "unparsed_cell" and f.status ==
    +"FAIL" for f in findings)
134      if table.kind not in SUPPORTED_KINDS or unparsed:
135          return "unsupported", counts
133 -    return ("check" if counts["FAIL"] or counts["WARN"] else "
    -ok"), counts
136 +    if counts["FAIL"] or counts["WARN"]:
137 +        return "check", counts
138 +    # numeric_matches_raw passes for any table with numbers, s
    +o it says nothing about accounting
139 +    evidence = sum(
140 +        1
141 +        for f in findings
142 +        if f.status in ("PASS", "PASS_ROUNDING") and f.check !
    += "numeric_matches_raw"
143 +    )
144 +    return ("ok" if evidence else "unverified"), counts
145
146
147  def skipped_pages(results: Sequence[PageResult]) -> pd.DataFra
     me:
```

Ran 1 shell command (ctrl+o to expand)

● Only that one edit had missed; the others did apply. Now the two real fixes, using the Edit tool. First, multiple footnote markers:

Read 1 file (ctrl+o to expand)

● Update(src¥fin_table_parser¥normalize.py)
  └ Added 11 lines, removed 7 lines
```
29  # It must be removed BEFORE NFKC: after folding, "※１2,121,360"
     would read as 12,121,360. Filers
30  # write the marker with full-width digits and the amount with A
    SCII digits; if both are ASCII and
31  # glued together the marker is taken to be a single digit (a fo
    otnote number above 9 is rare).
32 -_NOTE_MARKER = re.compile(r"^¥s*[※＊*]¥s*(?:([０-９]+)|(¥d)(?=¥
   -s|¥D))¥s*")
32 +_NOTE_MARKER = re.compile(r"^¥s*[※＊*]¥s*(?:([０-９]+)|(¥d)(?=¥
   +s|¥D))¥s*[,, 、]?¥s*")
33
34
35  def split_note_marker(raw: str) -> tuple[str, tuple[str, ...]]:
36 -    """``"※１2,121,360"`` -> (``"2,121,360"``, (``"※1"``,))."""
```

```
    m = _NOTE_MARKER.match(raw)
    if m is None:
        return raw, ()
    digits = nfkc(m.group(1) or m.group(2))
    return raw[m.end() :], (f"※{digits}",)
    """``※１2,121,360`` -> (``2,121,360```, (``※1```,)); se
    veral markers are all removed.

    ``※１,※２162,303``` is notes 1 and 2 and the amount 162,30
    3.
    """
    notes: list[str] = []
    rest = raw
    while (m := _NOTE_MARKER.match(rest)) is not None:
        notes.append(f"※{nfkc(m.group(1) or m.group(2))}")
        rest = rest[m.end() :]
    return rest, tuple(notes)


def parse_value(raw: str) -> Value | None:
    """Parse one printed cell. Returns None for an empty cell."
    ""
    body, notes = split_note_marker(raw)
```

Ran 1 shell command (ctrl+o to expand)

● Now the cost-of-goods-sold composition: closing stock and transfers to other accounts are subtracted in a total. I'll add that vocabulary and apply the sign in the component sum.

● Update(src¥fin_table_parser¥labels.py)
  └ Added 11 lines

```
    "株主資本等変動計算書",
)

# Lines printed as positive amounts that are SUBTRACTED inside the total they belong to, e.g. in a
# cost-of-goods-sold build-up: opening stock + purchases + production cost - closing stock.
SUBTRACTED_COMPONENTS = ("期末棚卸高", "他勘定振替高", "他勘定へ振替", "他勘定への振替")


def component_sign(label: str) -> int:
    """-1 for a line that is deducted within its total, else +1."""
    key = label_key(label)
    return -1 if any(s in key for s in SUBTRACTED_COMPONENTS) else 1


ASSETS_TOTAL = "資産合計"
LIABILITIES_EQUITY_TOTALS = {
    "負債純資産合計",
```

Read 1 file (ctrl+o to expand)

● Update(src¥fin_table_parser¥validate.py)
  └ Added 1 line, removed 1 line

```
                continue
            if not _has_value(total, col):
                continue
            expected = sum(_number(rows[p], col) for p in parts)
            expected = sum(component_sign(rows[p].label) * _number(rows[p], col) for p in parts)
            n_terms = sum(1 for p in parts if _has_value(rows[p], col))
            findings.append(
                _compare(
```

● Update(src¥fin_table_parser¥validate.py)
  └ Added 1 line, removed 1 line

```
from collections.abc import Sequence

from .hierarchy import parent_indices
from .labels import ASSETS_TOTAL, LIABILITIES_EQUITY_TOTALS, label_key
from .labels import ASSETS_TOTAL, LIABILITIES_EQUITY_TOTALS, component_sign, label_key
from .models import Finding, Row, Status, Table
from .normalize import nfkc
from .rules import check_formulas, check_ties
```

Ran 1 shell command (ctrl+o to expand)

● The comprehensive-income and standalone P&L pages now pass, but the cash flow statement fails because it spans two pages—formulas on page 1 reference targets on page 2, so verifying pages individually incorrectly flags page 1 as unverified. I need to verify by grouping pages of the same statement kind

```
  together rather than checking each page alone. · summary

● Update(src¥fin_table_parser¥ui_model.py)
  └ Added 38 lines
    144      return ("ok" if evidence else "unverified"), counts
    145
    146
    147 +def group_health(tables: Sequence[Table]) -> list[tuple[str, d
        +ict[str, int]]]:
    148 +    """Health of each table, judged together with the other pa
        +ges of the same statement type.
    149 +
    150 +    A statement can span pages (a cash-flow statement, a balan
        +ce sheet's two halves) and its checks
    151 +    use rows from several pages, so verification is per group
        +of tables of the same kind. A table
    152 +    is 'ok' only if its group passed at least one real account
        +ing check and failed none; a table
    153 +    with an unreadable cell or an unsupported kind is 'unsuppo
        +rted' regardless of its group.
    154 +    """
    155 +    from collections import defaultdict
    156 +
    157 +    from .validate import check_unparsed_cells
    158 +
    159 +    groups: dict[str, list[Table]] = defaultdict(list)
    160 +    for t in tables:
    161 +        groups[t.kind].append(t)
    162 +    group_state: dict[str, tuple[str, dict[str, int]]] = {}
    163 +    for kind, members in groups.items():
    164 +        findings = validate_tables(members)
    165 +        counts = summarize(findings)
    166 +        evidence = any(
    167 +            f.status in ("PASS", "PASS_ROUNDING") and f.check
        +!= "numeric_matches_raw"
    168 +            for f in findings
    169 +        )
    170 +        if counts["FAIL"] or counts["WARN"]:
    171 +            state = "check"
    172 +        else:
    173 +            state = "ok" if evidence else "unverified"
    174 +        group_state[kind] = (state, counts)
    175 +    out: list[tuple[str, dict[str, int]]] = []
    176 +    for t in tables:
    177 +        state, counts = group_state[t.kind]
    178 +        own_unparsed = any(f.status == "FAIL" for f in check_u
        +nparsed_cells(t))
    179 +        if t.kind not in SUPPORTED_KINDS or own_unparsed:
    180 +            state = "unsupported"
    181 +        out.append((state, counts))
    182 +    return out
    183 +
    184 +
    185  def skipped_pages(results: Sequence[PageResult]) -> pd.DataFra
         me:
    186      return pd.DataFrame(
    187          [{"Page": r.page_no + 1, "Reason": r.reason} for r in
         results if r.table is None]

  Ran 1 shell command (ctrl+o to expand)

● Background command "Install YomiToku into its own venv" completed (exit code
0)

● YomiToku has finished installing. First I'll switch the app to group-based
  health, then move on to the OCR workers.

● Update(src¥fin_table_parser¥app.py)
  └ Added 2 lines, removed 1 line
    144          type="primary",
    145      )
    146
    147 -health = {id(r): table_health(r.table) for r in shown if r.tab
        -le is not None}
    147 +_states = group_health([r.table for r in shown if r.table is n
        +ot None])
    148 +health = {id(r): state for r, state in zip(shown, _states, str
        +ict=True)}
    149  tab_names = [f"{STATUS_ICON[health[id(r)][0]]} {table_title(r)
         }" for r in shown]
    150  tabs = st.tabs([*tab_names, "Checks", "Pages"])
    151

● Update(src¥fin_table_parser¥app.py)
  └ Added 1 line, removed 1 line
    20      select_results,
    21      skipped_pages,
    22      table_frame,
    23 -    table_health,
    23 +    group_health,
```

```
     24      table_title,
     25  )
     26  from fin_table_parser.validate import summarize, validate_table
         s

  Ran 1 shell command (ctrl+o to expand)

● 155 pass. I'll resolve that lint with a clearer structure rather than a nested
  ternary.

● Update(src¥fin_table_parser¥ui_model.py)
  └ Added 2 lines, removed 5 lines
     167              f.status in ("PASS", "PASS_ROUNDING") and f.check
          != "numeric_matches_raw"
     168              for f in findings
     169          )
     170 -        if counts["FAIL"] or counts["WARN"]:
     171 -            state = "check"
     172 -        else:
     173 -            state = "ok" if evidence else "unverified"
     174 -        group_state[kind] = (state, counts)
     170 +        verified = "ok" if evidence else "unverified"
     171 +        group_state[kind] = ("check" if counts["FAIL"] or coun
         +ts["WARN"] else verified, counts)
     172      out: list[tuple[str, dict[str, int]]] = []
     173      for t in tables:
     174          state, counts = group_state[t.kind]

  Ran 1 shell command (ctrl+o to expand)

● The cold import of torch is slow on first use (it was moved to the
  background). I'll let it finish rather than poll.

  Read 1 file, ran 1 shell command (ctrl+o to expand)

● Lint, types, and all 155 tests pass, and YomiToku 0.15.1 (CPU torch build) is
  installed. Next I'll write the worker script that adapts it to my
  protocol—outputting JSON to a file, logging to stderr, and reporting load time
  separately—while tracking the model weight download for the ledger. · summary

● Write(tools¥ocr¥yomitoku_worker.py)
  └ Wrote 71 lines to tools¥ocr¥yomitoku_worker.py
     1 """YomiToku OCR worker. Runs inside ``.venv-yomitoku`` (not the m
       ain environment).
     3     python yomitoku_worker.py --out result.json page1.png page2.p
       ng ...
     4
       :
     6     {"engine": "yomitoku", "load_seconds": float,
     7      "pages": [{"seconds": float, "words": [{"text", "x0", "y0",
     8 Boxes are in image pixels, origin top-left.
     9 """
    … +67 lines (ctrl+o to expand)

  Ran 1 shell command (ctrl+o to expand)

● While that runs, the PaddleOCR worker (same protocol), to be tested once its
  install completes.

● Write(tools¥ocr¥paddle_worker.py)  └ Wrote 78 lines to tools¥ocr¥paddle_worker.py
     1 """PaddleOCR worker. Runs inside ``.venv-paddle`` (not the main e
       nvironment).
     3     python paddle_worker.py --out result.json page1.png page2.png
        ...
     4
       :
     6     {"engine": "paddle", "load_seconds": float,
     7      "pages": [{"seconds": float, "words": [{"text", "x0", "y0",
     8 Boxes are in image pixels, origin top-left.
     9 """
    … +79 lines (ctrl+o to expand)

● While the engines work, I'll re-check the UI with the new verification logic.
  The cover-page tables should no longer be ticked.

  Ran 1 shell command (ctrl+o to expand)

● Background command "Lint and test again; inspect the YomiToku API and Paddle
progress" completed (exit code 0)

● The tab list still reflects old behavior, likely because the running server
  has stale modules while fresh code passes in tests. I'll restart the server
  fully and re-check instead of assuming. · summary

  Ran 2 shell commands (ctrl+o to expand)

● Background command "Restart the UI server with the new config" failed with
exit code 255

  Ran 1 shell command (ctrl+o to expand)
```

● With a fresh server the cover-page tables (p2, p3) are marked [unsupported] as plain
  "Table", and only the genuine statements are [ok]. The earlier result was stale
  code. Let me look at the final default view.

  Read 1 file (ctrl+o to expand)

● That's the minimal UI I'd present, with empty blank cells and the Japanese
  statement name in the caption. Now I'm checking back on the YomiToku run
  (downloading model weights) and the PaddleOCR install. · summary

  Ran 1 shell command (ctrl+o to expand)

● Background command "Run the YomiToku worker on the sample page (first run
downloads models)" completed (exit code 0)

● YomiToku runs successfully end-to-end, processing 126 words in 23s on CPU. The
  9.9GB Hugging Face cache is larger than what YomiToku alone would need, so
  I'll check what was actually created during this session before claiming or
  deleting anything. · summary

  Ran 1 shell command (ctrl+o to expand)

● The 9.9 GB is mostly pre-existing speech-model cache (Whisper, pyannote)
  unrelated to this work—YomiToku only added two small model folders. I'll
  locate exactly where those weights landed so the ledger stays accurate.
  · summary

  Ran 1 shell command (ctrl+o to expand)

● Background command "Install PaddleOCR into its own venv" completed (exit code
0)

● PaddleOCR is installed, and I've noted YomiToku's two cached weight files
  (~290 MB total) so only those get removed later. Now running PaddleOCR on the
  sample page for the first time, which will trigger its own model downloads.
  · summary

  Ran 1 shell command (ctrl+o to expand)

● While PaddleOCR runs, I'll score YomiToku's output on the sample page right
  away. This is the first real OCR result through the production path.

  Ran 1 shell command (ctrl+o to expand)

● The production OCR run nailed it: all 43 rows and 74 amounts match perfectly,
  with correct labels, indents, row types, and accounting checks—only the column
  headers differ. I'll check how the engine read those headers next. · summary

  Ran 1 shell command (ctrl+o to expand)

● The OCR captured the date header correctly, but my regex was too strict about
  spacing around 年/日, causing dates to be dropped. I'll make the date parsing
  and header comparison whitespace-insensitive to handle this variation.
  · summary

● Update(src¥fin_table_parser¥normalize.py)
  └ Added 1 line, removed 1 line
    18 _NIL = {"-", "–", "‐", "−", "—", "ー"}
    19 _NEGATIVE_MARKERS = "△▲"
    20 _NUMBER = re.compile(r"¥d{1,3}(?:,¥d{3})+|¥d+")
    21 -_DATE = re.compile(r"(¥d{4})年¥s*(¥d{1,2})月¥s*(¥d{1,2})日")
    21 +_DATE = re.compile(r"(¥d{4})¥s*年¥s*(¥d{1,2})¥s*月¥s*(¥d{1,2})¥
       +s*日")
    22
    23
    24 def nfkc(text: str) -> str:

  Ran 1 shell command (ctrl+o to expand)

● Update(src¥fin_table_parser¥evaluate.py)
  └ Added 1 line, removed 1 line
    165   "statement": nfkc(table.statement) == nfkc(truth.state
         ment),
    166   "unit": (table.unit or "") == truth.unit,
    167   "revision": (table.meta.get("revision") or None) == tr
         uth.revision,
    168 -   "headers": [nfkc(h) for h in table.headers] == [nfkc(h
        -) for h in truth.headers],
    168 +   "headers": [label_key(h) for h in table.headers] == [l
        +abel_key(h) for h in truth.headers],
    169   "periods": list(table.periods) == list(truth.periods),
    170   }.items():
    171   r = ratio(f"table.{name}")

● Update(src¥fin_table_parser¥evaluate.py)
  └ Added 1 line, removed 1 line
    356   for name in ("statement", "unit", "kind"):
    357   if getattr(a, name) != getattr(b, name):
    358   diffs.append(f"{name}: {getattr(a, name)!r} != {ge

```
      tattr(b, name)!r}")
359 -    if [nfkc(h) for h in a.headers] != [nfkc(h) for h in b.hea
    -ders]:
359 +    if [label_key(h) for h in a.headers] != [label_key(h) for
    +h in b.headers]:
360          diffs.append(f"headers: {a.headers} != {b.headers}")
361      if a.periods != b.periods:
362          diffs.append(f"periods: {a.periods} != {b.periods}")
```

Ran 1 shell command (ctrl+o to expand)

● YomiToku is now perfect on the sample page at page level. Let me see how PaddleOCR is doing.

Ran 1 shell command (ctrl+o to expand)

● Background command "First PaddleOCR run on the sample page" completed (exit code 0)

● Found a known PaddlePaddle 3.x CPU bug on Windows causing PaddleOCR to fail due to a missing oneDNN operator implementation. I'll disable that acceleration in the worker (speed-only impact) and retry. · summary

● Update(tools¥ocr¥paddle_worker.py)
  └ Added 3 lines

```
49          use_doc_orientation_classify=False,
50          use_doc_unwarping=False,
51          use_textline_orientation=False,
52 +        # PaddlePaddle 3.x on Windows CPU fails inside oneDNN (
   +"ConvertPirAttribute2RuntimeAttribute
53 +        # not support ...") for these models; turning that acce
   +leration off changes speed only.
54 +        enable_mkldnn=False,
55      )
56      load_seconds = time.time() - t0
57
```

Ran 1 shell command (ctrl+o to expand)

● PaddleOCR now runs but is slow on this CPU setup (223 s for one page versus YomiToku's 23 s). Let me see its accuracy before judging.

Ran 1 shell command (ctrl+o to expand)

● PaddleOCR nails the numbers (98.6% cell F1) but misreads 15 of 43 Japanese labels and runs about 10× slower. Before drawing conclusions, I want to rule out a configuration issue, so I'll check its CPU-thread setting against the machine's core count. · summary

Ran 1 shell command (ctrl+o to expand)

● The machine's 16 cores rule out CPU starvation, so PaddleOCR's slowness likely comes from processing the full 2480×3508 px image without downscaling. I'll test capping the long side at 1920 px, which is standard practice and fair tuning for both engines. · summary

● Update(tools¥ocr¥paddle_worker.py)
  └ Added 4 lines

```
52          # PaddlePaddle 3.x on Windows CPU fails inside oneDNN (
    "ConvertPirAttribute2RuntimeAttribute
53          # not support ...") for these models; turning that acce
    leration off changes speed only.
54          enable_mkldnn=False,
55 +        # A 300 dpi page is 2480 x 3508 px. The default only en
   +forces a MINIMUM side, so the model
56 +        # would run at full size; the usual setting caps the lo
   +ng side (and is far faster).
57 +        text_det_limit_type="max",
58 +        text_det_limit_side_len=int(os.environ.get("PADDLE_DET_
   +SIDE", "1920")),
59      )
60      load_seconds = time.time() - t0
61
```

Ran 1 shell command (ctrl+o to expand)

● Capping the time to 90s kept accuracy the same, confirming it's not a configuration issue. Next I'll examine exactly what PaddleOCR misreads in the labels to see if it's fixable or an engine limitation. · summary

Ran 1 shell command (ctrl+o to expand)

● I've found that PaddleOCR's Japanese model misreads key classification terms with variant kanji (e.g., 資? instead of 資産) and merges a nil dash into labels, causing lost amounts—issues YomiToku doesn't have. I'll still run the full comparison across a varied, spread-out page selection from all documents to confirm this conclusion isn't based on a single page. · summary

● Update(src¥fin_table_parser¥ocr_eval.py)
  └ Added 6 lines, removed 1 line

```
66                  found_here += 1
```

```
     67                 if one_per_document:
     68                     break
     69 -    return pages[:limit] if limit else pages
     69 +    if limit and len(pages) > limit:
     70 +        # spread the choice over all documents instead of takin
        +g the first few (which would be
     71 +        # near-identical correction notices)
     72 +        step = (len(pages) - 1) / max(1, limit - 1)
     73 +        pages = [pages[round(i * step)] for i in range(limit)]
     74 +    return pages
     75
     76
     77  def edit_distance(a: str, b: str) -> int:
```

Ran 1 shell command (ctrl+o to expand)

● I've got six pages covering the different document types (correction notice, full-year and half-year summaries, two quarterlies, and the annual report with footnotes). I'm now running the full comparison in the background across 2 engines × 6 pages × 3 scan conditions, which will take a while on CPU.
· summary

Ran 1 shell command (ctrl+o to expand)

● While that runs (it's CPU-heavy, so I'll stick to light work), I'll bring the accounting-checks explanation up to date with everything the corpus taught me.

Ran 1 shell command (ctrl+o to expand)

● While the bake-off runs, I'll record the durable facts about this work for future sessions (the goal, the agreed working rules, and where things stand).

Listed 1 directory, wrote 3 memories (ctrl+o to expand)

● Everything else is in place, and the bake-off is the one remaining dependency. I'll set up a proper wait on its completion rather than poll.

● The bake-off's own completion notification is enough; no extra monitor needed. Meanwhile I'll record the measured disk footprint of everything installed, so the cleanup proposal is concrete.

Ran 1 shell command (ctrl+o to expand)

● The disk measurement is slow while OCR saturates the CPU. I'll write the Phase 2 work log now (the bake-off numbers get added when it finishes).

● Write(docs¥worklog¥phase2-worklog.md)
```
  ⎿  Wrote 60 lines to docs¥worklog¥phase2-worklog.md
      1 # Phase 2 work log
      2
        ). Order: A (statement-aware validation), B (band-free
      4 core), C (hybrid with OCR), D (evaluation), plus an upload UI bu
        ilt alongside. Status at the end of this entry:
        f"); D partly done (silver references, bake-off).
      6
      7 ## What was built
      9 | Step | Deliverable | Evidence |
     10 |---|---|---|
    … +103 lines (ctrl+o to expand)
```

● Everything that doesn't depend on the OCR comparison is finished and verified. I'm waiting on the bake-off (2 engines × 6 pages × 3 scan conditions on CPU, running in the background), and I'll pick up from its results when it completes.

Where things stand
- Validation (step A): the whole earnings filing passes 80 accounting checks with 0 failures. The income-statement false alarms are gone.
- Band-free parser (step B): it matches the band-based parser exactly on all 28 comparable pages, including the annual report with footnote markers.
- Upload UI: I ran it in a headless browser against the 19-page and 109-page files and fixed what I saw. A [ok] now requires a check that actually passed.
- OCR, first real results on the sample page:
  - YomiToku: all 74 amounts, 43 labels and 43 indents correct, in 23 s on CPU.
  - PaddleOCR: 98.6% on amounts and 65% on labels. It writes variant kanji (資? for 資産) and reads the nil dash as ー, and it is about 4× slower.
- Tests: 156 pass, with ruff and mypy clean.

When the bake-off finishes I'll write up the comparison, propose what to delete (with sizes) and wait for your OK before removing anything, and give you the final summary.

● Background command "Measure the disk footprint of everything installed" completed (exit code 0)

Read 1 file (ctrl+o to expand)

● Real sizes are in. Updating the ledger with them (and noting that ~/.cache/torch dates from July, so it isn't ours).

● Write(docs¥worklog¥installed-packages.md)
  └ Added 33 lines, removed 18 lines
```
 1  # Installed packages ledger
 2
 3 -Rule agreed with the user: large downloads are fine, but anythi
   -ng redundant is removed once it is no
 4 -longer needed, and nothing is deleted without telling them. OCR
   - candidates and the browser used for UI
 5 -checks live in **their own virtual environments** under the pro
   -ject root, so removing one is deleting one
 6 -folder and cannot break the main environment.
 3 +Rule agreed with the user: large downloads are fine, but anythi
   +ng redundant is removed once it is no longer
 4 +needed, and **nothing is deleted without the user's OK**. OCR c
   +andidates and the browser used for UI checks live in
 5 +their own virtual environments under the project root, so remov
   +ing one is deleting one folder and cannot break
 6 +the main environment. Model weights and caches live *outside* t
   +hose folders, so they are listed separately.
 7
 8 +Measured 2026-10-23.
 9 +
10  ## Main environment (system Python 3.12)
11
12  | Package | Why | Status |
13  |---|---|---|
12 -| pymupdf | read PDF text layer, drawings, render pages | keep
   -(was already installed; AGPL, fine for personal use) |
14 +| pymupdf | read PDF text layer, drawings, render pages (AGPL,
   +fine for personal use) | keep (was already installed) |
15  | openpyxl | write / read the Excel output | keep |
14 -| pyyaml | accounting rules file | keep (was already installed)
   - |
15 -| pandas | tables in the UI | keep (was already installed) |
16 -| streamlit | the upload-a-PDF UI | keep |
16 +| pyyaml, pandas | rules file, UI tables | keep (were already i
   +nstalled) |
17 +| streamlit 1.41.0 | the upload-a-PDF UI. Pulled in pyarrow, al
   +tair, pydeck, tornado, watchdog (about 0.3 GB) | keep |
18  | pytest, ruff, mypy | tests, lint, types | keep (dev tools) |
19  | fin_table_parser (editable install) | `python -m fin_table_pa
     rser ...` | keep |
20
21  `numpy`, `pydantic` were already present and are unused by this
     project.
22
22 -## Isolated environments (project root, all deletable)
23 +## Isolated environments (project root)
24
24 -| Folder | Contents | Why | Status |
25 +| Folder | Size | Contents | Purpose | Proposal |
26 +|---|---|---|---|---|
27 +| `.venv-yomitoku/` | 1.1 GB | yomitoku, torch (CPU build) | OC
   +R candidate 1 | keep if it wins the comparison |
28 +| `.venv-paddle/` | 926 MB | paddlepaddle, paddleocr, opencv |
   +OCR candidate 2 | remove if it loses |
29 +| `.venv-browser/` | 125 MB | playwright | headless browser for
   + UI checks | remove (UI check done) |
30 +
31 +## Caches and weights outside the project (created by this work
   +)
32 +
33 +| Location | Size | What | Proposal |
34  |---|---|---|---|
26 -| `.venv-paddle/` | paddlepaddle, paddleocr, opencv-python-head
   -less | OCR candidate 1 | to be evaluated; sizes below |
27 -| `.venv-yomitoku/` | yomitoku (pulls torch) | OCR candidate 2
   -(Japanese-specialised) | to be evaluated; sizes below |
28 -| `.venv-browser/` | playwright + chromium | headless browser t
   -o check the UI visually | delete after UI verification |
35 +| pip cache `%LOCALAPPDATA%¥pip¥cache` | 944 MB | downloaded wh
   +eels | purge (`pip cache purge`); only costs re-download time |
36 +| `%LOCALAPPDATA%¥ms-playwright` | 719 MB | Chromium for playwr
   +ight | remove together with `.venv-browser` |
37 +| `~/.paddlex` | 133 MB | PaddleOCR models | remove together wi
   +th `.venv-paddle` if it loses |
38 +| `~/.cache/huggingface/hub/models--KotaroKinoshita--yomitoku-*
   +` (2 folders) and their 2 blobs (98 MB + 192 MB) | 290 MB | Yom
   +iToku weights | keep if YomiToku wins |
39
30 -Sizes and the final keep / remove decision are recorded in the
   -OCR comparison report
31 -(`docs/reports/ocr-comparison.md`) once it exists. Removal comm
   -and for an environment:
32 -`Remove-Item -Recurse -Force .venv-<name>` (PowerShell) after t
   -he user agrees.
40 +**Not ours, do not delete:** the rest of `~/.cache/huggingface`
   + (about 9.6 GB: Whisper, pyannote, sentence-transformers,
41 +all dated July 2026 or earlier) and `~/.cache/torch` (314 MB, d
```

```
   +ated July 2026).
42
43 +## Licences
44 +
45 +* PyMuPDF: AGPL-3.0 (fine for personal use, matters only if dis
   +tributed).
46 +* YomiToku: open-source edition reported as non-commercial; fin
   +e here (personal use). Verify in its repository before
47 +  any commercial use.
48 +* PaddleOCR / PaddlePaddle: Apache-2.0.
49 +
50  ## Downloaded data
51
36 -`data/corpus/yamato/` (10 PDFs, about 4 MB): test documents fro
   -m the company's public IR page, see
37 -`data/corpus/README.md`.
52 +`data/corpus/yamato/` (10 PDFs, 3.3 MB): test documents from th
   +e company's public IR page, see `data/corpus/README.md`.
```

● The ledger now has real sizes, and it separates what this project created from what was already on the machine. I'm still waiting on the bake-off; I'll continue as soon as it reports.

● 2 background shell command tasks didn't finish before the previous session ended. Task ids: b2of157Op, bbjfq4jvs.

> continue what u were doing and finish it off please!

● The previous session ended before the bake-off finished, so I'll check what state things are in, then re-run it.

  Ran 1 shell command (ctrl+o to expand)

● Nothing was written and nothing's running yet—code and all 156 tests are intact. I'll now update the CLI to write the report after each engine finishes, so partial results are never lost during long runs. · summary

  Ran 1 shell command (ctrl+o to expand)

● Update(src¥fin_table_parser¥cli.py)
  ⎿ Added 10 lines, removed 4 lines

```
 95      pdfs = sorted(Path(args.corpus).glob("*/*.pdf"))
 96      pages = find_silver_pages(pdfs, args.limit)
 97      print(f"{len(pages)} silver pages: {[p.name for p in pages
    ]}")
 98 -    bake = run_bakeoff([found[n] for n in wanted], pages)
 99 -    report = render_markdown(bake, pages, {})
 98 +    from .ocr_eval import Bakeoff
 99 +
100 +    combined = Bakeoff()
101      Path(args.output).parent.mkdir(parents=True, exist_ok=True
     )
101 -    Path(args.output).write_text(report, encoding="utf-8")
102 -    print(report)
102 +    for name in wanted:  # one engine at a time; the report is
    + rewritten after each, so none is lost
103 +        part = run_bakeoff([found[name]], pages)
104 +        combined.results += part.results
105 +        combined.load_seconds.update(part.load_seconds)
106 +        Path(args.output).write_text(render_markdown(combined,
    + pages, {}), encoding="utf-8")
107 +        print(f"finished {name}; report updated", flush=True)
108 +    print(Path(args.output).read_text(encoding="utf-8"))
109      return 0
110
111
```

  Ran 1 shell command (ctrl+o to expand)

● The bake-off is running again, with the report now rewritten after each engine so nothing is lost. YomiToku goes first, then PaddleOCR, and I'll continue once it reports.

● The bake-off is running again, with the report now rewritten after each engine so nothing is lost. YomiToku goes first, then PaddleOCR, and I'll continue once it reports.

> /compact
  ⎿ Compacted
  ⎿ Read src¥fin_table_parser¥cli.py (155 lines)
  ⎿ Read docs¥worklog¥installed-packages.md (53 lines)
  ⎿ Read ..¥..¥AppData¥Local¥Temp¥claude¥c--Users-suggu-Projects-assignment¥654
    3be26-9de4-402d-ab84-a30111c72378¥tasks¥buajk9wz3.output (20 lines)
  ⎿ Read docs¥worklog¥phase2-worklog.md (61 lines)
  ⎿ Read ..¥..¥.claude¥projects¥c--Users-suggu-Projects-assignment¥memory¥feedb
    ack-working-rules.md (20 lines)

● Task "Re-run the OCR bake-off in the background" still running in background

> continue working please

Ran 1 shell command (ctrl+o to expand)

● The bake-off is still running, and no engine has finished yet, so there is no report. YomiToku runs first and its results will be written to docs/reports/ocr-comparison.md as soon as it completes. I'm waiting for the completion notice and will then write up the comparison and propose the cleanup list. I won't delete anything until you confirm.

> /advisor
  ⎿ Advisor: Opus 5.5
     Usage: /advisor <fable|opus|sonnet|off>

● Background command "Re-run the OCR bake-off in the background" completed (exit code 0)

Searched for 1 pattern, ran 2 shell commands (ctrl+o to expand)

● The UI has no explicit warning for OCR pages, so my log line overstated. I'll correct it.

Ran 1 shell command (ctrl+o to expand)

● The OCR bake-off has finished, and YomiToku is clearly the better engine. The full tables are in docs/reports/ocr-comparison.md, and the work log is updated. I didn't re-run the 156 tests or lint after this, since I changed no code.

The test used 6 balance-sheet pages, each read under three conditions: clean 300 dpi, a scan-like 150 dpi low-quality JPEG, and a 1° skew.

	YomiToku	PaddleOCR
Cell F1 (clean / scan-like / skewed)	97.2% / 92.8% / 97.3%	83.7% / 90.7% / 84.1%
Pages exactly right (of 6)	4 / 3 / 5	0 / 0 / 0
Label accuracy	98-99%	65-69%
Seconds per page	about 12	about 72

- Not proof of accuracy: the reference pages are text-layer parses that pass every accounting check, not human-made, and they all come from one company. The scan conditions are simulated.
- Checks miss OCR errors: the accounting checks flagged only some of the wrong YomiToku pages (1 of 2 clean, 1 of 3 scan-like). A page that passes the checks is therefore not guaranteed correct, because a misread kanji label slips past a totals check.
- UI gap: the UI doesn't yet mark OCR-read pages with a warning. That is a possible follow-up.

Proposed cleanup. I'll delete nothing until you say yes. Sizes are from my earlier measurement.

Remove	Size	Why
.venv-paddle	926 MB	PaddleOCR lost
~/.paddlex	133 MB	PaddleOCR models
.venv-browser	125 MB	browser checks of the UI are done
%LOCALAPPDATA%¥ms-playwright	719 MB	Chromium for those checks
pip cache (pip cache purge)	944 MB	only costs re-download time

That totals about 2.8 GB. I'd keep .venv-yomitoku (1.1 GB) and its two model files in the Hugging Face cache (290 MB). I won't touch the rest of your

Hugging Face or torch caches.

Using it. Start the UI with python -m fin_table_parser ui, which stays on localhost. Nothing is committed or pushed. The ground truth for the sample page was written by me and is still waiting for your review in docs/review/ground-truth-review.md.

Shall I go ahead with the deletions above?

● Background command "cd "C:/Users/suggu/Projects/assignment"; cat >> docs/worklog/phase2-worklog.md <<'EOF' ## OCR bake-off result (6 silver pages x 3 conditions, CPU only) Full tables: ../reports/ocr-comparison.md. | Engine | Cell F1 (clean / scan-like / skewed) | Pages exact (of 6) | Label accuracy | Char error | s/page | |---|---|---|---|---|---| | YomiToku | 97.2% / 92.8% / 97.3% | 4 / 3 / 5 | 98-99% | 1.2-3.6% | about 12 | | PaddleOCR | 83.7% / 90.7% / 84.1% | 0 / 0 / 0 | 65-69% | 4.6-5.7% | about 72 | Decision: YomiToku is the OCR engine (more accurate on Japanese labels, about 6x faster). PaddleOCR is not needed. Caveats: silver references, one company, simulated scan conditions. The accounting checks flagged only some of the wrong YomiToku pages (1 of 2 clean, 1 of 3 scan-like), so an OCR page that passes the checks is not proof it is right: the UI shows OCR pages with that warning in mind. Label errors are mostly kanji misreads, which a total-based check cannot see. EOF du -sh .venv-* 2>/dev/null" completed (exit code 0)

● That notification is only the earlier work-log append finishing. It isn't a reply from you, so I haven't deleted anything.

The work log already has the corrected wording about the UI, and I fixed it before that task completed.

I'm still waiting for your OK on the cleanup: .venv-paddle, ~/.paddlex, .venv-browser, the Playwright Chromium and the pip cache, about 2.8 GB in total. Say "go ahead" and I'll remove them and update the installed-packages ledger. You can also name which ones to keep.

> yeah u can remove the unnecessary ones. before we move on with UI. How far have we come with our goal? Like how much of our parser is completed as of now? please also explain the status of existing pipeline to me

Ran 2 shell commands (ctrl+o to expand)

● The cleanup is done. I removed .venv-paddle, .venv-browser, ~/.paddlex, the Playwright Chromium and the pip cache, about 2.8 GB in all. I kept .venv-yomitoku and its model files. The 156 tests still pass, and YomiToku is the only OCR engine the UI now lists. I recorded the removals in docs/worklog/installed-packages.md.

How far we've come

Your goal was to take a page of a Japanese balance sheet, possibly scanned, and get a correct table with labels, values, columns, hierarchy, units and signs, with no hard-coded coordinates. That works for the cases I tested, with real gaps below.

Reasonably done:
- Text-layer PDFs (pages with real text):
  - The sample page matches my hand-written ground truth except for 3 cells.
  - The whole company filing gives 80 accounting checks with 0 failures.
  - The ruled-band method and the ruler-free method agree on all 28 pages where both apply.
  - Results are the same from 0.4× to 3× page scale.
- Output: one Excel workbook per PDF with one sheet per statement, plus Validation and Metadata sheets. Signs are kept: △ is negative, — is nil, and footnote markers are stripped.
- Checks: the balance sheet must balance, totals must equal their components, and the income statement and cash flow have formula rules in rules/statement_rules.yaml. A table only gets a green tick if a real check passed.
- UI: you upload a PDF, pick pages, and get tables, checks and an Excel download. It stays on localhost.

Partly done:
- Scans: the pipeline automatically sends a page to OCR when the text layer is unusable. YomiToku gets about 97% of cells right on clean pages and 93% on scan-like pages. Only 3 to 5 of 6 pages come out fully exact.
- OCR safety net: the checks flagged only some of the wrong OCR pages, because misread kanji labels pass a totals check.
- UI: it doesn't yet mark OCR-read pages as less trustworthy.

Not done:
- Pages drawn with full grid lines (equity statement, notes) and statements with a note-number column.
- Normal-spacing wrapped labels where the label sits at the top of the row.
- Other companies' filings. Everything is tested on one company's documents, so layout variety is small.
- Real scanned PDFs. All scan conditions were simulated by degrading rendered pages.
- Your review of the ground truth, which I wrote myself. It is still pending in docs/review/ground-truth-review.md.

How the pipeline works

1. Triage: read the PDF. If the text layer is healthy, use it. If it is garbled or missing, or no table is found on a mostly-image page, render the page and use OCR.
2. Extract: a ruler-free method finds rows by text line spacing, columns by where numbers end on the right, and the header block above the first amount. When the page has coloured row bands, a band-based method does the same job. Every tolerance is a fraction of a length measured on the page, so nothing depends on absolute coordinates.
3. Interpret: labels are indented into a hierarchy, values are parsed with signs and footnotes, and the units and period are read from the header.
4. Validate: the page is classified (balance sheet, income statement and so on) and the matching checks run. Each gets a status of pass, pass with rounding, fail, warn or skip.
5. Output: Excel, shown in the UI with a health state per table.

For OCR pages, the words are first de-skewed, then go through the same ruler-free extractor, so there is one code path for text and scans.

What I'd suggest next

1. Test real scanned PDFs and other companies' filings, which is the biggest unknown. If you have any, send them.
2. Mark OCR-read pages in the UI as needing a human look.
3. Review the ground truth.

Which of these do you want first?

> Ok. is there any other upgrades we can do to our parser or not? if so then what are they.
Also please create a readme and update it along with the AGENT.md file!

Ran 1 shell command (ctrl+o to expand)

● Write(README.md)
  ⎿ Wrote 83 lines to README.md
     1 # Japanese financial-statement table parser
     3 Takes a PDF page that holds a Japanese consolidated balance shee
       t (layout may vary, may be a scan) and produces a
     4 structured table: row labels, values, columns, hierarchy, units
     5 hard-coded; every tolerance is a fraction of a length measured o
       n the page. Personal, non-commercial project.
     7 Output is Excel: one workbook per PDF, one sheet per statement,
       plus a Validation sheet (accounting checks) and a
     8 Metadata sheet.
    10 ## Status
    … +110 lines (ctrl+o to expand)

Ran 2 shell commands (ctrl+o to expand)

● No dev extra exists, so I'll fix the install lines, then update AGENT.md and the docs index.

● Update(README.md)
  ⎿ Added 3 lines, removed 2 lines
    25 Python 3.12. From the project root:
    26
    27 ```
    28 -pip install -e .[dev]   # pymupdf, openpyxl, pyyaml (+ pyt
       -est, ruff, mypy)
    29 -pip install streamlit   # only for the UI
    28 +pip install -e .   # pymupdf, openpyxl, pyyaml
    29 +pip install streamlit pandas   # only for the UI
    30 +pip install pytest ruff mypy   # only for development
    31 ```
    32
    33 OCR (optional, only for scanned pages) lives in its own virtual
         environment so it cannot disturb the main one:

Ran 1 shell command (ctrl+o to expand)
● I created README.md at the project root and updated AGENT.md and the docs index (docs/README.md). The README covers status, install, usage, how it works, layout, quality gates and caveats. I didn't run the tests; only lint ran and it was clean.

- AGENT.md: I only added a "Project context" section at the end and left your rules untouched. The section covers the goal, the no-hard-coded-coordinates rule, the code map, the quality gates, the install and deletion rules, localhost-only for the UI, treating corpus PDFs as untrusted, and the unreviewed ground truth. You said "update it", which I read as adding project context. If you meant something else, tell me.
- README install lines: pip install -e . plus separate lines for the UI and dev tools. The project has no dev extra.
- Docs index: it now points to the README and describes the OCR comparison as done.

Upgrades possible

I haven't started any of these. AGENT.md says to propose a plan and get your approval first, so tell me which you want and I'll write one.

Accuracy and trust (highest value):
1. Cross-check text layer and OCR. When a page has both, run both and compare. Disagreements show exactly where a misread kanji or digit is, which the totals check can't see.
2. Label vocabulary check. Compare OCR labels against a list of standard Japanese balance-sheet terms and flag near-misses (for example 繰延税金資産 misread by one character). This targets the main OCR failure.
3. Mark OCR pages in the UI as "needs a human look", and show the OCR confidence per cell.
4. Per-cell review in the UI. Click a cell to see the cropped page image beside it and fix it, with checks re-running live.

Coverage:
5. Grid-ruled pages and note-number columns. This covers the equity statement, notes and the 注記 column, which are currently unsupported.
6. Top-aligned wrapped labels with normal line spacing.
7. Other companies and real scans. This needs documents from you or more public filings. It is the biggest unknown, and the real test of whether the layout handling generalises.
8. Multi-column and multi-period pages (for example consolidated and non-consolidated side by side), and English-language statements.

Evaluation:
9. Independent ground truth. Compare against XBRL data from EDINET filings, since XBRL is machine-readable and neither mine nor the parser's. This would replace my self-written truth.
10. Your review of the existing ground truth, which costs you little and would validate what we have.

Engineering:
11. Speed: YomiToku takes about 12 s a page on CPU. A GPU, or OCR on only the page region, would help.
12. Packaging and git: a dev extra in pyproject, and a first commit once you approve. Nothing has been committed.
13. Continuous checks: a single script that runs tests, lint and mypy.

I'd pick 1, 2 and 3 first (they make scans trustworthy), then 7 and 9 (they show whether the parser generalises). Which do you want?

> ok first publish this current parser to a public repo in my github.
Once done u can start working on the upgrades and test them out to see how well it does.

Ran 5 shell commands (ctrl+o to expand)

● I'm preparing the first commit. I'm leaving the conversation transcript docs/chat.md out of the public repo, then I'll verify a fresh clone passes the tests before pushing.

Ran 4 shell commands (ctrl+o to expand)

● The parser is published at https://github.com/premsuggu/japanese-financial-table-parser, a public repo with 3 commits on main. I chose the repo name myself, so tell me if you want a different one.

What I left out of the public repo:
- Company PDFs: the two root PDFs and the 10 corpus PDFs are the company's filings, so I didn't redistribute them. data/README.md says where to put them.
- Skipped tests: a fresh clone gets 78 tests passing and 78 skipped, because those need the PDFs. With the PDFs, all 156 pass. I made the skips automatic.
- Chat transcript: docs/chat.md, your chat transcript, is excluded as personal.
- Local junk: output/, the .venv-* folders and the caches are excluded as well.

I scanned the files for your email, tokens and keys and found none. Commits are authored as premsuggu.

I'll start on the upgrades next, beginning with the three that make scans trustworthy: cross-checking text layer against OCR, the label vocabulary check, and marking OCR pages in the UI. Per AGENT.md I'll write the plan first, and I'll run the tests on each. I won't push any of it without your OK.

> yeah sure go ahead and start working on the upgrades.

● Per AGENT.md I need an approved plan before implementing, so I'm writing it first.

● Write(docs¥plans¥phase3-plan.md)
  ⎿ Wrote 65 lines to docs¥plans¥phase3-plan.md
    1 # Phase 3 plan: make scanned pages trustworthy (proposal, awaitin
      g approval)
    3 ## Problem
    4

```
     5 On OCR pages the accounting checks catch only some wrong pages (b
     6 label (kanji) or a digit error that keeps the totals consistent p
       asses silently. Goal: raise the share of wrong OCR
     7 pages that get flagged, without flooding right pages with false a
     8
     9 ## Scope (three upgrades, in this order)
    … +86 lines (ctrl+o to expand)
```

● The plan is written at docs/plans/phase3-plan.md, and I haven't implemented anything yet. AGENT.md requires your approval of a detailed plan first.

  What it covers:
  - U1, multi-read agreement: read a page more than once and flag the cells where the reads differ.
    - For a text-layer page, compare the text layer against OCR.
    - For a scan, read it twice at different resolutions.
    - It doubles OCR time (about 12 s a page becomes about 24 s).
    - Two reads by the same engine can make the same mistake, so this lowers the risk but doesn't remove it.
  - U2, label vocabulary check: warn when an OCR label is one character away from a known account name, with the suggested fix. It never auto-corrects.
  - U3, UI: each page shows whether it was read from the text layer or by OCR. OCR tables get a "review" state unless every check passed and the reads agreed.
  - How we'll judge it: I'll re-run the bake-off and report three numbers, before and after:
    - how many wrong pages get flagged;
    - how many right pages get flagged by mistake;
    - how many flagged cells were actually wrong.

    If the safety net barely improves, I'll say so and not claim the upgrade.

  I need three answers from you:
  1. Do you approve the plan, or want changes?
  2. Should U1 be opt-in only, which I recommend, or always on for scans?
  3. May I build the label list from the corpus filings? They come from one company, so it would only catch near-misses of common accounts. Or do you have a better source?

> Plan approved.
  as long as performance is improved we can have the changes u proposed.
  u can find other sources if needed.

  Ran 5 shell commands (ctrl+o to expand)

● Escaping slipped in the scripted edit; fixing with the Edit tool.

● Update(src¥fin_table_parser¥pdf_io.py)
  ⎿ Added 1 line, removed 1 line
```
     82      """The same picture at ``factor`` times the pixel size (JPE
         G stays JPEG, else PNG)."""
     83      pix = pymupdf.Pixmap(data)
     84      resized = pymupdf.Pixmap(pix, max(1, round(pix.width * fact
         or)), max(1, round(pix.height * factor)))
     85 -    if data[:2] == b"ÿØ":
     85 +    if data[:2] == b"¥xff¥xd8":
     86          return bytes(resized.tobytes("jpeg", jpg_quality=90))
     87      return bytes(resized.tobytes("png"))
     88
```

  Ran 3 shell commands (ctrl+o to expand)

● Write(src¥fin_table_parser¥crosscheck.py)
  ⎿ Wrote 102 lines to src¥fin_table_parser¥crosscheck.py
```
      1 """Read agreement: compare two independent reads of the same pag
      2
        or that keeps the totals consistent.
      4 Two reads of one page (text layer vs OCR, or OCR of the same pic
      5 the same mistake in the same cell, so a cell where they differ i
      6
        l amounts (then a label difference is
      8 a label misread), then by identical label (then an amount differ
      9 Whatever is left has no partner and is reported as a row seen by
     10 """
    … +108 lines (ctrl+o to expand)
```

  Ran 1 shell command (ctrl+o to expand)

● I'm building the read-agreement check and the OCR collection is running in the background. Next is a standard-label list for the vocabulary check, so I'm looking for an official source.

● Web Search("EDINET タクソノミ 勘定科目リスト jppfs 日本語ラベル ダウンロード 金融庁 2025年版")
  ⎿ Did 1 search in 7s

● Fetch(https://www.jpx.co.jp/equities/listed-co/disclosure-gate/term/index.html)
  ⎿ Received 0 bytes (403 Forbidden)

Ran 4 shell commands (ctrl+o to expand)

● The label list picked up note text, so I'm tightening it to statement tables and short labels only.

Ran 6 shell commands (ctrl+o to expand)

● I'm wiring the second read and label check into the pipeline and validation. Next I'll add imports and run lint, mypy and the tests.

Ran 4 shell commands (ctrl+o to expand)

● Writing unit tests for read agreement and the label check, plus a pipeline test with a simulated engine that makes a mistake on its second read.

Ran 6 shell commands (ctrl+o to expand)

● I'm adding the UI side: a "double-check" option, a new "review" state for OCR pages that weren't double-checked, and a "Read via" column in the Excel metadata.

Ran 5 shell commands (ctrl+o to expand)

● My replacement string had the same escaping issue; I'll fix it with the Edit tool.

● Update(src¥fin_table_parser¥ocr_eval.py)
  └ Added 1 line, removed 2 lines
    451   f"| {hits}/{flagged} |"
    452   )
    453   lines.append("")
    454 -   return "
    455 -".join(lines)
    454 +   return "¥n".join(lines)

Ran 1 shell command (ctrl+o to expand)

● The code for all three upgrades is in and tested (170 tests pass; lint and mypy are clean). The OCR reads for the evaluation are still being collected in the background, so I haven't measured anything yet. When they finish I'll run the safety-net evaluation. If it doesn't improve detection, I'll report that instead of claiming the upgrade.

● Background command "cd "C:/Users/suggu/Projects/assignment"; echo "data/cache/" >> .gitignore; python -X utf8 "C:/Users/suggu/AppData/Local/Temp/claude/c--Users-suggu-Projects-assignment/6543be26-9de4-402d-ab84-a30111c72378/scratchpad/collect.py" > /tmp/collect.log 2>&1; echo exit $?" completed (exit code 0)
Ran 5 shell commands (ctrl+o to expand)

● Status: tightening the comparison so print-format differences don't count as errors, then re-running the evaluation.

Ran 5 shell commands (ctrl+o to expand)

● Everything for the three upgrades is built, and I wrote the explanation doc 05-read-quality-checks.md (docs/explanations/05-read-quality-checks.md). A larger evaluation over all 22 reference pages is running in the background. I'll update the report, work log, README and docs index once it finishes, then re-run the tests.

On the first 6 pages, the existing checks catch 3 of the 4 pages that are really wrong. Adding the second read catches all 4, with 1 false alarm in 14 right pages. That sample is small.

● Background command "cd "C:/Users/suggu/Projects/assignment"; mv docs/reports/phase3-metrics.md docs/reports/phase3-second-read-size.md python -<<'EOF' def sub(p, old, new): s=open(p,encoding="utf-8").read() assert old in s,(p,old[:50]) open(p,"w",encoding="utf-8").write(s.replace(old,new,1)) sub("src/fin_table_parser/ocr_eval.py","RESCALES = (1.0, 0.75, 1.33)","RESCALES = (1.0, 0.75) # 1.33 was tried too: see docs/reports/phase3-second-read-size.md") c="src/fin_table_parser/cli.py" sub(c,"pages = find_silver_pages(pdfs, args.limit)¥n reads = collect_reads(","pages = find_silver_pages(pdfs, args.limit, one_per_document=not args.every_page)¥n reads = collect_reads(") sub(c,''' sf.add_argument("--limit", type=int, default=6)''','''
sf.add_argument("--limit", type=int, default=6) sf.add_argument("--every-page", action="store_true", help="every silver page, not one per document")''') EOF ruff format src -q; ruff check src -q --fix; mypy 2>&1|tail -2 python -X utf8 -m fin_table_parser safety-eval --limit 100 --every-page --cache data/cache/yomitoku_reads_all.json -o docs/reports/phase3-metrics.md > /tmp/safety_all.log 2>&1; echo exit $?" completed (exit code 0)

Ran 4 shell commands (ctrl+o to expand)

● The three upgrades are built and tested, and they catch more of the wrong OCR pages. Lint, mypy and all 170 tests pass.

Result (YomiToku, 22 test pages under 3 simulated scan conditions, 66 reads; 15 of the 66 reads are really wrong):

	Wrong pages caught	False alarms
Accounting checks alone (before)	10 of 15	none
+ read twice and compare	13 of 15 alone, all 15 with the checks	8 of 51 right pages flagged
+ label vocabulary	3 to 6 of 15 alone	included in the 8 above

- What was added:
  - Read twice and compare: reads the page a second time (a scan at 0.75× size, or OCR against the text layer) and flags cells where the reads differ. It is opt-in, and it doubles OCR time from about 12 s to about 24 s a page.
  - Label vocabulary: warns when an OCR label is one character away from a known account name, and suggests the fix without changing anything.
  - UI: a new "review" state for OCR pages that haven't been double-checked, a "Read twice and compare" checkbox, and "Read via" columns in the Excel metadata.
- Cost: 84% of the flags (81 of 97) point at a real error. About 1 right page in 6 gets flagged for a look.
- Second-read size: I tested 1.33× as well and kept 0.75×. 1.33× caught only 2 of 4 wrong pages on the first 6 pages, against 4 of 4.
- Caveats:
  - The 22 pages are 10 documents from one company, many near-identical.
  - The label list is built from that same company's filings. The 3-of-15 figure rebuilds it without the page's own document, but quarterly filings still share labels.
  - The scan conditions are simulated, and the reference parses are not human-made.
  - A flag shows where to look, not which read is right.
- Other fixes: I found and fixed a crash where a banded layout with unusable columns raised a bare error instead of a normal "unsupported page" result. The scoring here counts print-format differences (a space inside a number) as harmless, so it is more lenient than the earlier OCR comparison.

The details are in phase3-worklog.md (docs/worklog/phase3-worklog.md) and phase3-metrics.md (docs/reports/phase3-metrics.md). I also added an explanation doc, updated the README and docs index, and included the label list in the package data.

Nothing from this phase is committed or pushed. Do you want me to commit and push it to the GitHub repo?

Next in the upgrade list are other companies and real scans, which is where the label list and layout handling are really tested. After that come the per-cell review in the UI and the independent ground truth from XBRL. Which do you want first?

> Now proceed with the next upgrades and test them as well. I have commited and pushed the changes as well.

● Web Search("決算短信 PDF 連結貸借対照表 IFRS 連結財政状態計算書 2025年3月期 IR資料")
  └ Did 1 search in 6s

● Web Search("EDINET API v2 XBRL download without API key public csv financial statements")
  └ Did 1 search in 6s

● Write(docs¥plans¥phase4-plan.md)
  └ Wrote 57 lines to docs¥plans¥phase4-plan.md
     1 # Phase 4 plan: other companies, other layouts (proposal, awaiti
     2
     3 ## Problem
     4
        mostly means "works on this company's layout".
     6 The unsupported list (note-number column, grid-ruled pages, top-
     7 untested because the corpus never shows those layouts. Goal: fin
       d out how the parser really behaves on other
     8 companies, then fix what breaks, with numbers before and after.
    10 ## Steps
    … +74 lines (ctrl+o to expand)

● I've written the next plan at docs/plans/phase4-plan.md and haven't started on it, because AGENT.md requires your approval of the plan first.

The plan: so far the parser has only been tested on one company's filings, so the next step is to see how it behaves on other companies.
1. Corpus and baseline: download 10 to 15 public statements from other

companies, covering IFRS statements, pages with a note-number column, and different units. Run the parser unchanged and report what works and what breaks, so we have an honest "before" number.
2. Fixes: repair whatever the baseline shows, one failure at a time. Each starts with a failing test, and tolerances stay relative to the page.
3. Independent ground truth: compare against XBRL values the parser never touched, or against a few pages you review by hand.
4. More realistic scans: add blur, uneven lighting and noise to the simulated scans, then re-run the OCR safety-net evaluation on the new companies.

Per-cell review in the UI and speed work are left out of this phase unless you want them.

Risks: some company sites block downloads (the JPX site returned 403 for me), so I'll list which files I couldn't get. If a layout needs a design change rather than a patch, I'll bring you the options before building anything large.

I need three answers from you:
1. Do you approve the plan, or want changes?
2. For the independent ground truth: EDINET's free API needs a key registered to your email, which I can't do for you. Will you register one, or should I use the hand-check instead?
3. Do you have any real scanned PDFs, from any company? Even one would test the OCR route better than my simulation.

> proceed

Ran 1 shell command (ctrl+o to expand)
└ Interrupted · What should Claude do instead?

> Ok. Upgrades which need external api and all u don't need to do them. Just proceed with what all can be done. We need complete our parser ASAP

● I'll skip anything needing downloads or external APIs and focus on offline work: generating synthetic test pages for unsupported layouts, then building fixes. Starting with a page generator to find where the parser breaks.
· summary

Ran 1 shell command (ctrl+o to expand)

● My earlier scratch file inspect.py shadows the standard library; renaming it.

Ran 16 shell commands (ctrl+o to expand)

● Only a test expectation was wrong (the single-column page prints the first figure, 1,000). Fixing it and running the whole suite.

Ran 16 shell commands (ctrl+o to expand)

● Let me check the degraded image looks plausible, then start the OCR reads for the new "photo-like" condition in the background.

Read 1 file, ran 2 shell commands (ctrl+o to expand)

● Offline layout work is done and tested (184 tests pass): note-number column, wrapped labels, two-panel account form, IFRS totals, minus signs and unit forms. The OCR reads for a harder "photo-like" scan condition are running in the background. I'll write up the results when that finishes.

● Background command "cd "C:/Users/suggu/Projects/assignment"; cp docs/reports/phase3-metrics.md docs/reports/phase3-metrics-3conditions.md; python -X utf8 -m fin_table_parser safety-eval --limit 100 --every-page --cache data/cache/yomitoku_reads_all.json -o docs/reports/phase4-safety-with-photo-condition.md > /tmp/safety4.log 2>&1; echo exit $?" completed (exit code 0)

Ran 5 shell commands (ctrl+o to expand)

● The header split didn't reliably help (OCR also drops characters in this synthetic header), so I'm reverting it and recording the limitation instead.

Ran 2 shell commands (ctrl+o to expand)

● I fixed the layout gaps I could reproduce, but nothing has been tested on a real filing from another company. I skipped the downloads and the XBRL ground truth, as you asked, and used synthetic pages instead. All 184 tests pass, and lint and mypy are clean.

Now supported, each with a regression test (see 06-layout-variants.md (docs/explanations/06-layout-variants.md)):
- Note-number column (注記): it used to become a bogus value column. Note numbers are now kept per row and shown in the UI and in a new "Note ref" column in the Excel file.
- Labels wrapped over two lines with the amounts level with the first line: the second line used to become its own row.
- Two-panel account form (勘定式), assets left and liabilities right: this used to give silently scrambled rows. Both panels are now read and joined.
- IFRS statements: the balance check now recognises 負債及び資本合計 and its variants.

- Number and unit forms: minus signs and dashes (−1,234) read as negatives,
  and more unit-line forms are accepted ((単位 千円), （千円）).

Harder scan test. I added a photo-like condition (blur, noise, uneven light,
slight skew) and re-ran the OCR safety net on the 22 corpus pages. Results are
in phase4-safety-with-photo-condition.md
(docs/reports/phase4-safety-with-photo-condition.md).
- Photo-like scans: every page had at least one error, and about 93% of values
  were right. The checks flagged 21 of the 22 pages. One page with a single
  wrong item was not flagged by anything, so the safety net is not complete.
- Other conditions: the clean, scan-like and skewed numbers are unchanged from
  last time. All 7 wrong pages on clean scans, and all wrong pages in the
  other two conditions, were flagged.

Known gaps (also in the work log (docs/worklog/phase4-worklog.md)):
- Merged headers: when I ran YomiToku on the synthetic pages, it merged or
  dropped characters in two-line column headers, so the reported periods were
  wrong on 4 of the 5 pages. I tried a fix and reverted it because it didn't
  work reliably.
- Grid-ruled pages: reading these by OCR gives wrong cells with no check
  failing. The UI shows them as "unverified", not ticked.
- Real filings: the synthetic pages show each layout feature works on its own.
  They don't show that other companies' filings will parse.

The README and docs index are updated. The new changes are uncommitted.

To keep going, I'd need either real filings or scans from other companies,
which you'd have to provide or allow me to download. Without those, the
remaining offline work is the per-cell review in the UI and speed. Which do
you want, or should I stop here?

> GT review against parser: All correct. I reviewed the actual PDF page against
all 43 rows in your ground-truth sheet.
I have also commited and pused the latest changes.
and I believe that everything else is also ready for the final parser
submission.

I would like to run a final test by myself in the UI.

Ran 2 shell commands (ctrl+o to expand)

● Thanks for reviewing the ground truth. I've recorded it as human-verified in
AGENT.md, the README and my notes.

Running the UI

1. Open a terminal in the project folder, C:¥Users¥suggu¥Projects¥assignment.
2. Start the UI:
python -m fin_table_parser ui
3. Open http://localhost:8501 in your browser. It only listens on your own
   machine. Stop it with Ctrl+C in the terminal.

What to try

Test: Sample page
How: Upload balance_sheet_after_correction.pdf
What you should see: One table with 43 rows and a [ok] tab. Checks passed, none
failed.

Test: Whole filing
How: Upload complete_pdf.pdf, scope "All financial statements"
What you should see: Several tabs (balance sheet, income statement and so on),

with checks passing.

Test: Annual report
How: Upload data/corpus/yamato/S100YMK7/S100YMK7.pdf (109 pages)
What you should see: The balance-sheet pages with footnote markers. It takes a

little longer.

Test: Your own PDF
How: Upload any Japanese statement PDF
What you should see: If a page isn't supported, the Pages tab says why.

Test: Scan or OCR
How: In the sidebar set "Scanned pages" to yomitoku, tick "Force OCR on every
page"
What you should see: Slower (about 12 s a page). Tabs show [review] "review" because

OCR pages aren't double-checked.

Test: Double-check
How: Also tick "Read twice and compare"
What you should see: About twice as slow. Differences between the two reads
appear in the Checks tab.

For a quick OCR test, use "Pages", type 1, tick "Force OCR", and pick a
single-page PDF.

How to read the screen

- Tab icons:
  - [ok] a real accounting check passed.
  - [review] read by OCR, so look at it yourself.
  - [check] a check failed.
  - [unverified] nothing could be verified.
  - [unsupported] the layout isn't supported.
- Checks tab: every check with its status. Tick "Only problems" to see just the failures.
- Download buttons: the Excel file has one sheet per statement, plus Validation and Metadata sheets. Each table also has a CSV button.
- Page image: shown next to each table, so you can compare by eye.

Limits to keep in mind

- Grid-ruled pages (such as the equity statement) and layouts from other companies haven't been tested on real filings.
- OCR can merge two-line column headers, so periods may be wrong on scans.
- A passing check doesn't prove an OCR page is right. That's why those pages show [review].

If anything looks wrong, tell me the file and page number and I'll look into it.